NEBENAN | Detaillierte Roadmap ab GitHub-Stand

V1.0 – alle geplanten Systeme ausführlich | Giant + Worm | 9. Oktober 2026

Redaktionsregel: Alle noch zu entwickelnden Features werden wie in der ursprünglichen Designbeschreibung ausführlich erklärt. Was die GitHub-README des Branches nebenan-physik bereits beschreibt (Voronoi-Bruch, Box3D-Unterbau, Schutt-/Stützsystem, Stadtstreaming, bestehende Benchmarks, Save/Load für Gebäude), wird nicht nochmals erläutert. Das Vorhandensein der Basis bedeutet nicht, dass sämtliche neuen Systeme bereits laufen.

01 | Spielform und 1.0-Ziel

Nebenan bleibt ein physikbasiertes Action-Sandboxspiel mit separaten, dichten Maps, kein kontinuierliches Open-World-Spiel. Zwei Hauptformen: freier Sandboxmodus (Fahrzeuge, Tools, Powers, Figuren und Szenarien ausprobieren) sowie Missionen/Challenges (Ziele, Regeln, Wertung, Wiederholung). Der Spieler soll verschiedene Systeme kombinieren, statt vorgefertigte Zerstörungssequenzen abzuspielen.

1.0 festgelegt: Bewegung, physischer Kampf, Skate-/Bail-Mechaniken, Hooks, Fahrzeuge, Elemente mit zwei Modi, Terrain-Graben/-Zerstörung, MLS-MPM-Wasser, GPU-Feuer/Rauch, spielbare Maps und Missionen. Giant und Worm werden hier als neue Spielmodi vollständig spezifiziert; ob beide Launch-Pflicht sind, sollte vor dem Feature-Freeze bestätigt werden. Split-Screen/LAN sind erwünscht, der 1.0-Termin wurde nicht endgültig festgelegt.

02 | Spielerkontrolle – Warframe x Overgrowth x Mirror’s Edge

Bewegungsgefühl: Schnell, fließend, richtungsabhängig und sofort kontrollierbar. Geplant sind Sprint, Slide, Bullet-Jump/direktionaler Sprung, Aim-Glide, Wall-Run, Wall-Jump, Vault, Klettern, Luftlenkung und akrobatische Übergänge. Geschwindigkeit soll über Kombinationen und gutes Timing entstehen; nicht bei jedem Wechsel automatisch auf null zurückfallen.

Technische Umsetzung: Normalbewegung über kinematischen oder hybriden Character Controller mit Capsule-/Shape-Casts, Oberflächen-Normalen, Bewegungszuständen und kontrollierter Beschleunigung/Reibung. Aktive Ragdoll nur bei schweren Treffern, Stürzen oder Bails. Wandkontakt, Bodenneigung, Kanten, Deckenkollision, bewegliche Plattformen und zerstörte Auflageflächen müssen während jedes Übergangs neu bewertet werden.

Akzeptanz: Slide → Bullet-Jump → Aim-Glide → Wall-Run → Hook-Swing → sichere Landung in einem Fluss; nachvollziehbare Kamera, variable Eingaben, möglichst wenig Input-Latenz.

03 | Nahkampf, Schwertkampf und physische Reaktionen

Overgrowth-Vorbild: Schläge, Kicks, Lufttritte, Foot-Slams, Ausweichen, Konter, Grapples, Würfe und kontrollierte Landungen. Gegner reagieren nicht nur mit Treffereffekten, sondern mit Impuls, Haltung, Umkippen, Taumeln und Ragdoll; Treffergeometrie und Angriffswinkel sind entscheidend.

Schwertvariante: Physisch lesbare Hiebe, Blöcke, Paraden und Klingenbindungen; eigene Energie-/Plasmaklingen sind eine Option. Keine bloß frei teleportierenden Trefferzonen. Treffer eines Körpers oder einer Waffe können an das gemeinsame Impact-System der Welt übergeben werden.

Balancing: Wind-up, Reichweite, Trefferfenster, Recovery, Momentum und Kontrolle in der Luft bilden Gegenplay. Die Körperphysik darf Warframe-Schnelligkeit nicht in schwerfälliges Puppet-Steuern verwandeln.

04 | Skate 3: Skateboard, Tricks, Bail und Stunts

Fahrbare Skateboards mit Geschwindigkeit, Lean/Balance, Lenkung, Sprung-/Trickimpulsen, Rampen, Grinds als später abstimmbare Spezialfälle und physischem Bail. Beim Crash wechselt die Figur in einen kontrollierten Ragdoll-Zustand; danach Recovery zurück in den Character Controller. Ob vollständige Board-Simulation oder Hybridmodell verwendet wird, bleibt eine Engineeringentscheidung.

Gemeinsame Welt: Bordsteinkanten, zerstörte Rampen, Fahrzeugkontakte und herausgebrochene Trümmer verändern Fahrbahn und Stuntverlauf. Challenges können Tricks, Distanz, Kombos und kreative Crash-Szenen bewerten.

05 | Hooks und Seilschwingen – Standardausrüstung

Schießen des Hooks auf valide Ankerpunkte; Seillänge, Spannung, Schwingen, Ablösen, optionales Auf-/Abspulen und Impulserhaltung. Verbindung zu ruhenden und beweglichen Objekten; wenn ein Anker zerstört wird oder sich löst, muss das Seil korrekt reagieren. Auf Wunsch Grapple-Zug auf leichtere Objekte, ohne den Spieler bei jeder Interaktion unkontrolliert herumzuschleudern.

Technik: Seil als distanzbeschränkte Constraint mit robuster maximaler Dehnung und stabiler Zeitintegration; Kollisions- und Ankerberechtigungen sauber definieren. Testfälle: Wall-Run in Schwingen, Anker an fallendem Trümmer, zwei schnelle Richtungswechsel, Splitscreenkamera.

06 | Fahrzeuge – Burnout-inspirierte Crash-Physik

Fahrzeuge mit Beschleunigung, Lenkung, Bremsen, Gewichtstransfer, Sprüngen, Überschlägen und Umweltinteraktion. Crash-Schaden soll Geschwindigkeit, Winkel und getroffenes Bauteil berücksichtigen. Die Fahrbarkeit kommt vor physikalischer Perfektion: erst ein überzeugendes Auto, danach Fahrzeugklassen.

Schaden: Leichtes Blechschadenmodell durch gestufte Mesh-/Materialvarianten und lokale Deformation; schwere Treffer können Teile lösen, Räder beeinträchtigen und Energie in die Umgebung übertragen. Volle kontinuierliche Finite-Elemente-Deformation ist keine Voraussetzung. Fahrzeuge können Gebäudeteile treffen, durch beschädigte Wände fahren oder von Gebäudeteilen getroffen werden.

Spielerwechsel: Ein- und Aussteigen, Fahrt → Bail → Fußbewegung, Kamerawechsel und Missionen für Stunts/Kollisionen. Wichtige Tests: Mehrfach-Crash, Fahrzeug unter Trümmern, hohe Geschwindigkeiten, zerstörter Untergrund.

07 | Elementarformen und Angriff/Verteidigung

Bedienidee: Ziffern 1, 2, 3, 4, 5, 6 … wählen Elementformen oder Fähigkeitsslots; T schaltet zwischen aggressiven und defensiven Fähigkeiten. Die genaue Zuordnung von Nummern zu Formen oder Skills ist noch offen; das sollte vor der Control-Implementierung final entschieden werden. Wechsel müssen schnell sein und dürfen Parkour-/Kampfcombos nicht unnötig unterbrechen.

Element (Vorschlag)	Aggressiv / Defensiv / Mobilität
Feuer	Flammenstoß, Hitze-/Explosionswelle / Hitzeabwehr, Gegenfeuer / Hitze-Dash.
Wasser	Druckstrahl, Wasserstoß / dämpfende Barriere, Löschwirkung / Wasser-Surf.
Erde	Felswurf, Bodenschlag, Spike / Wand, Rüstung, Verankerung / Steinrampe.
Luft	Druckstoß, Werfen / Geschossablenkung, Sturzabfederung / Luftdash.
Blitz	Ladung, elektrische Treffer / elektrischer Konter, Unterbrechung / Schnellimpuls.

Element (Vorschlag)	Aggressiv / Defensiv / Mobilität
Eis	Geschosse, Frostflächen / Eisbarriere, Reibungskontrolle / Eisslide.

Wichtig: Diese sechs Elemente sind ein konkret ausgearbeitetes Beispiel, keine endgültig fixierte Auswahl. Fähigkeiten sollen möglichst dieselben Kräfte, Wärme-, Material-, Flüssigkeits- und Terrain-Schnittstellen verwenden wie sonstige Spielmechaniken. Individuelle Cooldowns/Kosten und Combobalance müssen getestet werden.

08 | Wasser – GPU MLS-MPM/APIC

Festgelegter Ansatz: GPU-basiertes Moving Least Squares Material Point Method (MLS-MPM) mit Particle-/Grid-Transfers; APIC-artige Geschwindigkeitsübertragung als betrachtete bzw. integrierbare Technik. Ziel: dynamisches Strömen, Fluten, Spritzen und Impulsaustausch mit Körpern und Gelände, ohne eine vollständige Zellberechnung über die gesamte Map.

Architektur: Partikel tragen Masse/Bewegung; ein temporäres Gitter löst Kräfte und Geschwindigkeiten; Rücktransfer auf Partikel. Aktive Bereiche, begrenzte Partikelbudgets, Schlafen/Inaktivschalten ruhiger Regionen und adaptive Detailstufen sind zwingende Performancehebel. In der Nähe von Gebäudekollisionen sind robustes Coupling und stabile Impulsübertragung entscheidend.

Optional: SWE mit HLLC + MUSCL nur als ergänzende günstige Fern-/Großflächenlösung für langsame oder ruhige Fluten, nicht als Ersatz für den ausgewählten MLS-MPM-Kern. Test: Wasser durch zerstörte Wand, durch gegrabenen Graben, Auftrieb/Schub auf Debris und Löschen von Feuer.

09 | Feuer und Rauch – sparse GPU Eulerian Combustion

Festgelegte Richtung: Sparse 3D-Gitter/Voxel für aktive Feuer- und Rauchregionen auf der GPU. Felder: Temperatur, Brennstoff, Sauerstoff, Rauch-/Gasdichte, Geschwindigkeit und Auftrieb. Transport, Zündung und Verbrauch müssen so zusammenwirken, dass Feuer räumlich überspringt und sich nicht nur als zufälliger Dekoeffekt ausbreitet.

Hybridoption: GPU-Cellular-Automata für günstige Zündung/Brandausbreitung kombiniert mit Eulerian-CFD für Strömung, Rauch und Auftrieb. Das ist ein diskutierter Architekturvorschlag; die konkrete Auslegung muss im Prototyp gemessen werden.

Kopplung: Feuer wärmt Materialien über Zeit, kann Verbindungsmaterial schwächen und reagiert auf neue Luftwege durch Zerstörung. Wasser entzieht Wärme, löscht, verdrängt und kann verdampfen. Nicht jede Partikelwolke braucht vollständige CFD: visuelle Rauchpartikel und simulierte Felder voneinander trennen. Tests mit Innenraumbrand, offener Wand, Wasserstrahl und starker Zerstörung.

10 | Terrain-Graben und Geländezerstörung

Lokales Graben, Schneiden, Wegsprengen, Abtragen und Tunneln als 1.0-Basis. Bisher diskutiert: Mesh-Boolean-/CSG-Schnitte, gezieltes Detach/Remove sowie Voronoi-/Polygon-Nachbearbeitung. Der Wurm benötigt außerdem entlang einer Bewegungskurve fortlaufend erzeugte Tunnelquerschnitte.

Technische Risiken: Schnelle kollisionsgültige Remeshes, saubere Übergänge an Fundamenten, keine kleinen kaputten Collider, zuverlässige Speicherdaten für Geländeänderungen. Grenzen von Sichtgeometrie und Physikgeometrie explizit definieren. Testfälle: grabbarer Hang, Loch unter Fahrzeug, Tunnel unter Haus, Explosionskrater und spätere Flutung.

11 | Giant gegen Fighter – asymmetrischer Spielmodus

Spielablauf: Ein Spieler steuert einen massiven Giganten, der zweite einen deutlich kleineren, extrem beweglichen Fighter. Der Giant kontrolliert Raum und Umgebung mit Schlägen, Sweeps, gezielten Stomps,

Greifen und gegebenenfalls Werfen; der Fighter überlebt durch Dash, Wall-Run, Haken, Timing, Deckung und Gegenangriffe.

Standardsteuerung – Targeting statt komplizierter Gliedmaßen-Puppe: Crosshair wählt Punkt oder Objekt. Linke/rechte Maustaste kontrollieren die jeweilige Arm-Attacke, Q/E die Fußangriffe; Angriff halten = aufziehen/zielen, Loslassen = ausführen. Angriffsart hängt von Zielhöhe/-richtung ab: Frontschlag, Seitwärtsschwung, Abwärtshieb, Tritt oder Stampfen. Eingabebelegung ist Vorschlag, kein festes Mapping.

Physik: Ziel → erreichbare IK-Bahn → aktive Joint-Motoren mit Drehmoment-/Tempo-Limits → reale Limb-Kollision → Impulse auf Fighter, Fahrzeuge, Gelände und Bauwerke. Kein garantiertes Animation-Hit. Assistierte Körperbalance hält den Giganten spielbar; Standfläche, Schwerpunkt und Fußkontakte begrenzen extreme Aktionen.

Gegenplay: Vorwarnende Körpersprache/Wind-up, begrenztes Retargeting nach Angriff-Commit, lesbare Arcs, Recovery nach massiven Treffern. Drei Zielmodi: direkt, prädiktiv (ohne Auto-Hit) und freier Umgebungspunkt. Fehlschläge sind erwünscht: verfehlte Schläge demolieren möglicherweise das Haus dahinter. Greifen von Autos, Blöcken oder Spielern später erweiterbar.

Prototyp-Abnahme: Ein Giant gegen einen Fighter auf einer kleinen Map; horizontaler Armschwung, gezielter Boden-Stomp, stabile Körperbalance, klar ausweichbare Angriffe und echte Gebäude-/Terrain-Kollisionen.

12 | Worm gegen Fighter – spielbarer Riesenwurm

Spielablauf: Spieler A lenkt einen riesigen Wurm, der unterirdisch gräbt, Fundamente unterhöhlt, auftaucht und durch Strukturen bricht. Spieler B muss überleben oder Ziele erfüllen, unterstützt durch Parkour und Hook. Das asymmetrische Prinzip ist Masse, Untergrundkontrolle und Überraschung gegen Mobilität und Wahrnehmung.

Fortbewegung: Kopf wird direkt gesteuert; Körpersegmente folgen einer geglätteten Bahn mit begrenzter Krümmung, Kollision und Verzögerung. Nicht jedes Segment muss uneingeschränkt als eigener Ragdollkörper laufen. An Oberfläche und unterirdisch unterschiedliche Kamera-/Steuerhilfen erwägen.

Tunnel: entlang der Kopfbahn eine Kapsel-/Spline-basierte Volumenoperation, lokale Gelände-Neuvernetzung und konsistente Kollisionsupdates. Durchbrechen von Straßen, Unterhöhlung von Fundamenten, Sinklöcher und das Herausstoßen von Schutt. Mit Wasser später verbunden: Tunnel leiten/füllen Ströme.

Fairness: Vibrationen, sichtbare Verformung, Risse oder Bodengeräusche warnen vor dem Auftauchen; ein klar lesbares Ausbruchfenster erlaubt Dodges. Prototyp: lenkbarer Kopf/Körper, ein Tunnel, ein Ausbruch, Gebäudeschaden und eine einfache Gegenmission.

13 | Sandbox, Missionen, Maps und UI

Sandbox: freie Auswahl von Szene, Figur/Spielmodus, Tools, Elementen, Fahrzeugen und physikalischen Parametern. Spielwelt-Reset, reproduzierbare Szenario-Starts, Debug-Statistiken und Spawn-Tools. Maps werden als kompakte, dichte Playgrounds gebaut: Stadtblock, Industrie, Baustelle, Skate-Areal, Hochhaus usw.

Missionen: Demolitionsziele (mit Einschränkungen), Parcours-/Stunt-Score, Kampf- und Fluchtziele, Giant-/Worm-Runden. Regeln, Zielmarker, Pause, Neustart, Erfolgszustand, Punkte/Zeiten und wiederholbare Herausforderungen. Erst je ein vollständiger Modus mit klarem Ende, dann Varianten.

14 | Gemeinsame Integration und technische Leitplanken

Einheitliche Interaktionsschicht: Player, Giant, Worm, Fahrzeuge, Wasser, Feuer und Elemente erzeugen gemeinsame Ereignisse für Impuls, Materialkontakt, Schaden, Geometrieänderung und Zustandswechsel.

Keine separaten Sonder-Destruction-Systeme pro Fähigkeit. Simulationen erhalten feste Substeps/Budgets, wohldefinierte Zuständigkeit für Kollision und Synchronisation.

Rendering: Sichtbarkeitsprüfung, LOD/HLOD, Chunk-Rendering, Instancing/Batching, effizienter Geometrie-Upload, Schatten-/Materialkosten, Staub und Audio. Leistungsziel nicht nur reine Physikzeit, sondern Spielend-Framezeiten mit Wasser, Feuer, Figur und Fahrzeug gleichzeitig. CPU/GPU getrennt messen; p95/p99, Speicher und Spikes überwachen.

Spielbarkeit/Release: Kamera, Controller/Keyboard-Maus, Optionen, Speicherstände für Spielstatus, Fehlertoleranz, QA, reproduzierbare Benchmarks, Package/Installer, rechtliche Prüfung von Dependencies und eigene Assets. Split-Screen und LAN wurden angedacht, sind für Release noch offen und müssen deshalb separat entschieden werden.

15 | Empfohlene Build-Reihenfolge ab jetzt

Meilenstein	Konkrete Lieferung / Test
M1: vertikaler Gameplay-Slice	Spielercontroller + Springen/Slide/Wall-Run + Hook in einer Map, stabil neben bereits existierender Destruction.
M2: physischer Action-Loop	Kicks, Trefferreaktionen, Ragdoll/Bail, kleines Skate-Setup, grundlegende Enemy-Reaktion.
M3: Fahrzeuge	Ein voll fahrbares Fahrzeug, Crash-/Schadensdarstellung, zuverlässig im Kontakt mit Umgebung.
M4: Terrain	Grab-/Krater-/Tunnel-Änderungen inkl. Kollision, Fundamenten, Save/Load.
M5: Wasser & Feuer	GPU MLS-MPM-Wasser und sparse GPU combustion in separaten Benchmark-Szenen, dann gekoppelt.
M6: Elemente	Wählbare Formen/Slots + T-Stance; je ein nützlicher Angriffs-/Defensivskill mit echter Weltwirkung.
M7: Giant + Worm	Je eine spielbare asymmetrische Testarena, telegraphierte Fähigkeiten und abnehmbare Gegenplay-Tests.
M8: Inhalte + Shipping	Sandbox, Missionsloop, Karten, Renderoptimierung, Audio/UI, Tests, Veröffentlichung.

16 | Nicht Teil der hier erneut erklärten Arbeit

Die bereits in der Repository-README dokumentierten Einzelheiten der vorhandenen Destruction-Library bleiben dort die verbindliche technische Beschreibung. Diese Roadmap wiederholt sie nicht; neue Systeme müssen sich über klar definierte APIs damit verbinden. Weitere Maps, neue Kräfte, FPV-Drohnen und zusätzliche Minigames können Updates sein. Die Reihenfolge ist ein Vorschlag, keine behauptete Implementierung oder feste Deadline.

17 | Destruction: Restanker, Buckling und Kollaps

Ergänzung vom 10. Oktober 2026 · Rekonstruktion der gemeinsam ausgearbeiteten Designregeln vom 29. September und 7. Oktober. Dieser Abschnitt ergänzt die Roadmap um das gewünschte Kollapsverhalten; er behauptet keinen bereits implementierten Stand. Die bisherigen fünf Seiten bleiben unverändert.

17.1 | Auslöser und die 90/10-Regel

Keine separate Buckling-Funktion: Buckling bezeichnet hier das entstehende Verhalten, keine klassische Knicksimulation und keine eigene Funktion, die das Haus gezielt verbiegt oder rotiert. Die ungefähr 10 % Restmaterial werden ebenfalls gebrochen, aber nicht weggesprengt. Sie bleiben zunächst stehen; dadurch kippt oder knickt das Haus automatisch durch die vorhandene Körperphysik, Schwerkraft und Kontakte. Anker-/Scharnierbereich beschreibt diesen Rest, keinen eigens erzeugten Hinge-Joint.

Als konkrete Designregel wurde ein Kollaps bei ungefähr **40 % zerstörter Etage** festgehalten. Frühere Varianten lagen bei etwa 40–50 %; der Schwellwert bleibt abstimmbar. Beim Auslösen werden ungefähr **90 % des noch verbleibenden Materials** freigesetzt bzw. weggesprengt. Ungefähr **10 % des verbleibenden Materials** bleiben als gebrochener Anker-/Scharnierbereich, an dem der Rest bzw. die darüberliegende Struktur kippen kann.

Rechenbeispiel: Ursprünglich 100 %, davon 40 % zerstört → 60 % verbleiben. Davon werden 54 Prozentpunkte freigesetzt; 6 Prozentpunkte der ursprünglichen Etage bleiben als Scharnierbereich. Die 10 % sind somit kein Anteil der ursprünglichen Gesamtetage.

Die Regel ist eine bewusst akzeptierte spielerische Näherung. Der Designwert „40 % zerstört“ darf nicht mit einem Support-Trigger wie „50 % tragende Wände verbleiben“ gleichgesetzt werden. Materialverlust und verbleibende Tragstruktur benötigen getrennte Bezugsgrößen.

17.2 | Einseitige Zerstörung

Liegt der Schaden überwiegend auf einer Seite, sitzt die Hinge-/Ankerkante auf der **stärksten überlebenden Seite**. Der kleine Rest soll dort zusammenhängend bleiben. Die übrigen Teile werden freigesetzt; der noch zusammenhängende obere Bereich knickt bzw. kippt **zur zerstörten oder nicht mehr unterstützten Seite**. So entsteht Sprengung mit anschließendem Buckling statt eines gleichförmigen Verschwindens.

17.3 | Gleichmäßig verteilter Schaden

Ist die Etage gleichmäßig durchlöchert, dominiert das Spreng-/Collapse-Verhalten. Die ungefähr 10 % Restmaterial können nach der akzeptierten Regel **zufällig gewählt** werden; eine künstlich bevorzugte starke Seite ist dann nicht erforderlich. Die Auswahl darf nicht mit der Einseitigkeitsregel verwechselt werden: einseitig bedeutet gezielter zusammenhängender Anker, gleichmäßig bedeutet verteilten Kollaps mit zufälligem Rest.

17 | Knickverhalten und Umsetzung

17.4 | Böden, Decken und Wände

Böden/Decken - lokaler Rampenkollaps: Wird von unten ein ausreichend großes Loch in einen Deckenbereich geschossen, der weit genug von den äußeren Deckenrändern entfernt ist, kann ein lokaler, zusammenhängender Bereich als Rampe nach unten klappen. Nicht die gesamte Decke wird aktiviert: Die umliegende Deckenfläche bleibt fest und stabil. Ein Treffer nahe dem äußeren Deckenrand löst keine automatische Rampenbildung aus; dort greift nur die normale lokale Voronoi-Zerstörung.

Geometriebedingung: Die ausgewählte lokale Platte muss genügend Länge und Bewegungsfreiheit besitzen, damit ihre freie Kante beim Herunterkippen den darunterliegenden Boden oder eine tragfähige Auflage erreichen kann. Ist das geometrisch nicht möglich, wird kein Rampenmodus ausgelöst. Der Rampenbereich wird aus benachbarten Voronoi-Stücken als steifer, zusammenhängender Rigid Cluster gehalten; seine inneren Verbindungen bleiben intakt.

Hinge-artiges Verhalten ohne künstliches Gelenk: Nur an einer geeigneten Anschlusskante zwischen lokalem Rampenbereich und fester Decke werden einige Voronoi-Stücke beziehungsweise ihre Verbindungen freigegeben. Wenige räumlich zusammenhängende Restverbindungen bilden eine nachgiebige Bruchzone. Ihre Auswahl kann unter den gültigen Kandidaten vereinfacht bzw. zufällig erfolgen. Schwerkraft, reale Kontakte und das schrittweise Nachgeben oder Brechen der Restverbindungen lassen die Platte als Ganzes kippen. Dafür muss die Bruchzone relative Bewegung physikalisch zulassen; eine vollständig starre Kopplung würde keine Drehung erzeugen.

Stabiler Endzustand: Sobald die freie Rampenkante den Boden oder tragfähigen Schutt erreicht, können Auflagekontakte und verbleibende Restverbindungen eine begehbare Rampe stabilisieren. Weitere Treffer oder Überlastung können die Bruchzone lösen und die Rampe einstürzen lassen. Eine Bodenberührung allein garantiert noch keinen sicheren Halt: Kontakt und Reibung müssen das Abrutschen verhindern.

Wände: Die bisherige Regel bleibt bestehen. Größere Wandbereiche können steif bleiben, während eine kleine Restzone an der Bruchkante geschwächt wird. Schwerkraft und Kontakte bestimmen Kippen, Hängenbleiben oder Abreißen.

18 | Parallax Occlusion Mapping

Ergänzung vom 11. Oktober 2026 | Render-System, noch zu implementieren

18.1 | Ziel und Einsatz

Parallax Occlusion Mapping (POM) erzeugt mittels Heightmap und blickwinkelabhängiger UV-Verschiebung wahrgenommene Tiefe auf weiterhin flachen Mesh-Oberflächen. Einsatz insbesondere für Ziegel, Putz, Asphalt, Fugen, Betonrisse, Fassaden und abgenutzte Materialien. Ziel ist hohe Oberflächendetailtiefe ohne entsprechende Geometrie- oder Physiklast.

18.2 | Technische Umsetzung

Materialshader: Tangent-Space-View-Vektor, Heightmap, Ray-Marching durch die Höhenstruktur und anschließende UV-Korrektur; lineare/binary Verfeinerung nur wo nötig. POM wird mit vorhandenen Normal-, Roughness- und Albedo-Maps gekoppelt. Verdeckte Details können sich perspektivisch gegenseitig überlagern. Selbstverschattung ist optional und abhängig vom Qualitätsprofil.

18.3 | Performance, Distanz und Grenzen

Materialabhängige Sample-Budgets mit dynamischen Stufen nach Blickwinkel, Bildschirmgröße und Entfernung. Nahbereich POM, mittlere Distanz reduzierte Samples, Fernbereich reguläres Normal Mapping. Frühzeitiges Culling, Texture-Streaming sowie Debug-Ansichten für Height/UV/Sample-Kosten vorsehen. Stark schräge Blickwinkel begrenzen, um Artefakte zu vermeiden.

Wichtig: POM verändert weder Mesh-Silhouette noch Collider oder tragende Voronoi-Geometrie. Einschusslöcher, tatsächlich aufbrechende Wände, Schutt und Deckenkanten bleiben echte Geometrie des Zerstörungssystems; POM stellt nur deren Materialoberflächen dar.

18.4 | Zusammenspiel mit Destruction und Abnahme

Frisch freigelegte Bruchflächen dürfen passende eigene Material-/Heightmap-Sets verwenden. POM muss auf bewegten Rigid-Clustern, Schutt und intakten Gebäuden konsistent wirken. Abnahmetest: Ziegelwand und gerissener Asphalt in flachem Blickwinkel; Wechsel POM ↔ Normal Mapping ohne störendes Flackern; umfangreiche Zerstörung bei kontrollierter GPU-Framezeit.

19 | Skeleton-first Character Pipeline

Automatische Erstellung animationsfertiger Low-Poly-Figuren, noch zu implementieren

19.1 | Verbindlicher Erzeugungsablauf

Zuerst wird das Skelett (Armature/Rig) erzeugt, danach die Körpergeometrie um dieses Skelett herum. Körperform, Proportionen und Gelenkregionen werden von den Bones bestimmt - kein zuerst frei modellierter Körper, der nachträglich notdürftig geriggt wird. Die Pipeline ist für Humanoide, Giants und geeignete Kreaturenvarianten ausgelegt; der Worm braucht ein eigenes segmentiertes Rig.

19.2 | Skelettkonstruktion

Aus einem Archetyp werden Bone-Hierarchie, Körpermaße, Root/Pelvis, Wirbelsäule, Hals, Kopf sowie Arm-, Hand-, Bein- und Fußketten aufgebaut. Definierte Gelenkzentren, lokale Bone-Achsen, Rest-Pose (A- oder T-Pose), Gelenkgrenzen und konsistente Namen bilden die verbindliche technische Grundlage. Finger, Gesichts- und Twist-Bones nur dort einsetzen, wo Animation und Spielmechanik sie tatsächlich erfordern.

19.3 | Low-Poly-Mesh um die Bones erzeugen

Anhand der Bone-Segmente und anatomischer Querschnitte wird eine zusammenhängende, animationsspezifische Low-Poly-Hülle aufgebaut. Topologie folgt Gelenkbeugungen: gezielte Edge-Loops an Schulter, Ellbogen, Hüfte, Knie, Handgelenk und Sprunggelenk; ausreichendes Volumen an Beugezonen, wenige Polygone auf starren Körperflächen. Keine beliebige Dreiecksverteilung oder überflüssige Subdivision. Silhouette und Deformation sind wichtiger als uniforme Polygonverteilung.

19.4 | Skinning und Qualitätsprüfung

Mesh wird automatisch an das bestehende Rig gebunden. Bone-Weights anhand von Knochenabstand, Körperteilzugehörigkeit und Gelenkzone initialisieren, normalisieren und an kritischen Übergängen glätten; Einflusszahl pro Vertex begrenzen (z. B. 4). Selbstüberschneidungen, offene Kanten, invertierte Normalen und extrem schmale Dreiecke prüfen. Automatische Pose-Tests für Kniebeuge, Armschwung, Sprint, Sprung, Hocken, Stomp und Ragdoll-Übergänge decken Deformationsfehler auf.

19.5 | Animation, IK und Physik

Ein gemeinsames benanntes Skeleton-Schema ermöglicht Animation-Retargeting und Übergänge zwischen Keyframe-Animation, IK und physischer Ragdoll. Physikkörper und Joint-Constraints werden vom Rig abgeleitet, nicht aus jedem Render-Polygon. Für Giant-Steuerung müssen Zielpunkt-IK, motorisierte Gliedmaßen und Balance die gleiche Bone-Hierarchie nutzen. Das visuelle Skinning darf Simulationskörper nicht destabilisieren.

19.6 | Game-ready Export und LOD

Ausgabe als optimiertes Skinned Mesh mit UVs, Materialslots, Skin-Weights, Bind-Pose, Rig, eindeutiger Skalierung und definierter Vorwärtsachse. Polybudget je Figurentyp festlegen statt fixe Werte für alle Wesen; LOD0 für Nahkampf, vereinfachte Skinned-LODs für Entfernung und gegebenenfalls Bone-Reduction für weit entfernte NPCs. Optional Export in glTF 2.0 oder das native Engine-Assetformat; Animationen und Physics-Assets werden separat testbar gehalten.

19.7 | Abnahme und Build-Reihenfolge

Ein frisch generierter Charakter muss unmittelbar ohne manuelles Nachriggen laufen, springen, treten, greifen und in eine Ragdoll wechseln können. Keine sichtbaren Gelenkrisse, starken Volumenverluste oder Mesh-Explosionen in Extremposen. Zuerst einen humanoiden Prototyp komplett von Bone-Erzeugung bis Animation/Export testen, anschließend auf Giant/Kreaturen erweitern.

19.8 | Integration in bestehende Roadmap

Diese Pipeline ist technische Voraussetzung für den Spielercontroller und physischen Kampf (M1/M2) sowie für Giant und Worm (M7, dort archetypspezifisch). POM ist ein Rendering-Feature, das nach dem Gameplay-Prototyp priorisiert und im Renderoptimierungs-Meilenstein M8 profiliert wird. Bestehende Destruction-Regeln in Abschnitt 17 bleiben unverändert.

20 | Gebäude-Streaming, Physikaktivierung und Cache

Ergänzung vom 11. Oktober 2026 | nächste Implementierungsschritte | Branch physik-5

Ziel: Wiederholtes Erzeugen und Entfernen von Häusern reduzieren, ohne alle Kollisions- und Zerstörungsdaten einer riesigen Stadt dauerhaft in Box3D zu halten. Die bestehenden Zerstörungs-, Speicher- und Wiederherstellungsfunktionen bleiben Grundlage. Dieser Abschnitt ist eine geplante Optimierung, kein bereits implementiertes Feature.

20.1 | Ausgangspunkt und gemessene Annahmen

In der Demo besteht die große Stadt aus 256 × 256 = 65.536 Häusern. Laut Quellkommentar würde vollständiges Laden etwa 24 GB beanspruchen (Schätzung, kein neuer Benchmark). Pro Streaming-Aufruf sind aktuell maximal zwei Loads, vier Unloads und ein Save eines beschädigten Hauses vorgesehen. Der Kamera-Laderadius liegt standardmäßig bei 100 m, Entladen beginnt erst jenseits von Radius + 20 m. Bewegte Bodies halten nahe Häuser für 300 Physikschritte aktiv.

Intakte Häuser werden bei geeignetem Abstand und nach nbDestructible_CanUnload entfernt; beschädigte, ruhende Häuser werden über nbSaveDestructibles gespeichert, zerstört und bei Bedarf über nbLoadDestructibles wiederhergestellt. Neu geladene Häuser durchlaufen PrepareCityHouse und nbStartCreating / nbFinishCreating. Die Initialszene lädt bereits Häuser in Kameranähe. Nicht voraussetzen, dass jeder Physikschritt tatsächlich neue Häuser baut: die Budgets und Aufrufhäufigkeit getrennt messen.

20.2 | Zielarchitektur: drei getrennte Ebenen

Ebene A - persistente Stadtbeschreibung: Position, Seed, Gebäudetyp, intakt/beschädigt, gespeicherter Zustand sowie Reichweite kompakt für sämtliche Häuser halten. Keine vollständige Physikwelt-Instanz pro Haus erforderlich.

Ebene B - vorbereitete Gebäudedaten: möglichst unveränderliche Piece-Definitionen, Öffnungen, Voronoi-Zellen, Nachbarschaften, Kollisionshüllen und weitere teure Vorberechnungen wiederverwendbar vorhalten. Cache-Einträge nach Typ/Seed/Geometrieparametern eindeutig schlüsseln und mit begrenztem RAM-Budget verwalten.

Ebene C - aktive Box3D-Welt: nur Häuser in relevanter Spiel- und Kollisionsnähe instanziieren. Intakte Bereiche bevorzugt statisch, getrennte bewegte Teile dynamisch simulieren; ruhenden Schutt über vorhandenes Freeze-System behandeln. Kamera-Sichtbarkeit, Physik-Relevanz und Speicherresidentheit dürfen verschiedene Radien/Budgets haben.

20.3 | Cache statt vollständiger Neuberechnung

Zuerst instrumentieren, welche Zeit bei wiederholtem Laden für PrepareCityHouse, Voronoi-/Hull-Vorbereitung, nbStartCreating, nbFinishCreating, Shape-Anlage und Render-Mesh-Upload anfällt. Die Pipeline in „geometrisch vorbereitet“ und „in Physikwelt instanziiert“ auftrennen, sofern die Schnittstellen dies zulassen.

Als ersten Schritt einen LRU-Cache für vorbereitete intakte Gebäudedefinitionen testen. Keine laufzeitgebundenen Body-/Shape-IDs in die wiederverwendbare Definition übernehmen. Bei gleichen Archetypen möglichst gemeinsame immutable Daten referenzieren, sofern Seeds, Türen/Fenster und Bruchmuster das erlauben. Beschädigte Gebäude benötigen weiterhin ihren individuellen Zustand einschließlich Trümmern.

20.4 | Aktivierung statt permanenter Vollsimulation

Für überschaubare Maps optional alle Häuser einmalig instanziieren und ihre intakten Kollisionskörper dauerhaft statisch halten. Erst freigesetzte Rigid-Cluster oder Trümmer benötigen dynamische Bodies. „Schlafen“ allein ersetzt keine räumliche Begrenzung: statische Shapes belegen weiterhin Speicher und Broadphase-Strukturen.

Bei großen Maps Streaming beibehalten, aber unnötige Wechsel verhindern: Hysterese zwischen Lade- und Entladeradius, längere Haltefristen, Priorität für nahende bewegte Objekte, vorzeitiges Laden entlang Bewegung und harte Budgets gegen Frame-Spitzen. Für sehr hohe Geschwindigkeiten muss der Vorladebereich die Geschwindigkeit und die verbleibende Erstellungszeit berücksichtigen.

20.5 | Zustandsübergänge und Korrektheit

Zustände explizit definieren: unloaded → prepared/cached → pending → active intact → active damaged → saved dormant → restored. Pro Haus nur eine laufende Erstellungs-/Wiederherstellungsoperation; stabile Identität trotz neuer Generations-IDs. Asynchrone Ergebnisse vor dem Einbau auf Gültigkeit und überholte Anfragen prüfen.

Beschädigung, Bond-Zustand, Position und Geschwindigkeit freier Körper, ruhender Schutt sowie Kontakte, die für sicheres Speichern relevant sind, müssen beim erneuten Laden konsistent bleiben. Bestehendes CanUnload-/CanSave-Verhalten darf nicht durch aggressiveres Entladen umgangen werden. Kein Haus entfernen, solange aktive Kollision, Instabilität oder relevante Auflagebeziehungen verloren gingen.

20.6 | Konkrete Benchmarkmatrix

In identischen Release-Builds Szenen mit 100, 300 und 1.000 Häusern vergleichen: (A) alles vorab laden, anschließend statisch halten; (B) bestehendes Streaming; (C) Streaming mit vorbereitetem Cache. Zusätzlich die 65.536-Häuser-Stadt mit ihrem realistischen Bewegungsprofil als Skalierungstest laufen lassen.

Szenarien: stehende Kamera, langsames Bewegen, schneller Flug/Fahrt, Umkehren am Radiusrand, wiederholte Route, Schuss/Explosion, Kollaps und ruhender Schutt, Rückkehr zu beschädigtem Haus. Pro Fall Warm- und Cold-Cache unterscheiden und mehrmals reproduzierbar wiederholen.

20.7 | Messgrößen und Entscheidungskriterien

Erheben: mittlere und p95/p99-Framezeit, einzelne CPU-Zeiten für Physik-Step/Streaming/Erzeugen/Speichern/Wiederherstellen, maximale Frame-Spikes, RAM-Peak und dauerhaft belegten RAM, aktive/statische/schlafende Bodies, Shapes, Broadphase-Last, Cache-Hits/Misses, Ladeverzögerung, Anzahl Lade-/Entladevorgänge und sichtbare/geometrische Artefakte.

Entscheidung: Permanente Physikinstanzen für kleinere dichte Maps nur übernehmen, wenn Speicher und Standbild-/Destruction-Framezeiten vertretbar sind. Für sehr große Welten vorbereitete Daten persistent oder gecacht halten und physische Instanzen räumlich streamen. Keine universelle Grenzzahl ohne Messung behaupten: Bruchstückdichte und Zielhardware ändern das Ergebnis stark.

20.8 | Reihenfolge der Implementierung und Abnahme

Schritt 1: Profiler-Zähler und reproduzierbare Szenarien einbauen. Schritt 2: Baseline von physik-5 erfassen. Schritt 3: einmaliges Laden für kleine Maps als separat schaltbaren Modus implementieren. Schritt 4: Cache für vorbereitete Gebäude, ohne Save/Load-Verlust, prototypisch integrieren. Schritt 5: dynamische Ladegrenzen/Prioritäten und Hysterese prüfen. Schritt 6: Matrix erneut messen und je nach Map-Größe Modus wählen.

Abnahme: Keine verschwundenen Kollisionen bei schneller Bewegung; beschädigte Häuser und Schutt nach Hin-/Rückweg identisch; kein mehrfacher House-Load; kein dauerhaft wachsender Cache/RAM; vorhersagbare p95/p99-Spikes und stabile Zerstörungsabläufe. Die bisherigen Abschnitte 1–19 bleiben unverändert.

21 | NVIDIA Blast und PhysX: Quellcodevergleich statt Enginewechsel

Ergänzung 12. Oktober 2026 | Recherche- und Benchmarkaufgabe | Referenzbranch: physik-8

21.1 | Zuständigkeiten sauber unterscheiden

Nebenan ist nicht Box3D und Box3D ist nicht Blast. Die korrekte Gegenüberstellung lautet: Nebenans eigene Zerstörungsbibliothek entspricht funktional in Teilen dem NVIDIA Blast SDK (Bruchstücke, Bonds, Verbindungsgraph, Schaden, Abtrennung, Actor-Splitting). Das modifizierte Box3D übernimmt die Rolle eines Rigid-Body-Solvers, vergleichbar mit PhysX CPU (Breitphase, Kontakte, CCD, Impulse, Constraints, Schlafzustand). Blast selbst bietet weder eine eigene Rigid-Body- noch eine Grafik-/Kollisionssimulation. Das bestehende Nebenan-System vereint beide Ebenen über seine APIs.

21.2 | Bereits vorhanden – nicht erneut als Optimierung vorschlagen

Voronoi-Vorzerlegung und lokale Laufzeitverfeinerung, flächenabhängige Bonds, lokaler Stützgraph, Abtrennung in Rigid-Cluster, dynamische Box3D-Körper, Kollisionsschaden, Multi-Worker-Fraktur, deterministische Einbaureihenfolge, statischer Schutt mit lokalem Auftauen, Save/Load und Streaming. Ebenfalls dokumentiert sind Batch-Shape-Insertion, Hull-Sharing ohne doppelte Geometrie, Kontakt-Prefetching, Worker-Schwellen, Solver-Cache-Optimierungen und bereits gemessene/verwarfene Ansätze in docs/Optimierungen.md. Vor jeder Idee aktuelle Implementierung und Messhistorie prüfen.

21.3 | Konkreter Claude-Forschungsauftrag

1. Lies die tatsächlichen Implementierungen von Nebenan/Box3D und NVIDIA Blast/PhysX CPU, nicht nur Readmes oder Videos. Belege technische Unterschiede mit Datei-, Funktions- und möglichst Commit-Verweisen.
2. Vergleiche Blast: Bond-Datenstrukturen, Damage-Pipeline, inkrementelle Komponentenbildung, ActorSplit, Speicherzugriffe, Job-Scheduling, Profiling und Fragment-/Graphverwaltung. Beachte, dass Blast und Nebenan nicht alle strukturellen Regeln gleich modellieren.
3. Vergleiche PhysX CPU: Broadphase, konvexe Narrowphase, Contact-Persistence, Manifold-Cache, Constraint-/Island-Solver, Task-Verteilung, Schlafmanagement, Allokationsmuster und große ruhende/aktive Körpermengen.
4. Identifiziere nur neue, lokal integrierbare Ideen. Ordne jede Idee der verursachenden Profiling-Kostenstelle zu; bereits implementierte oder widerlegte Lösungen explizit aussortieren.
5. Bestehende Architektur bleibt erhalten. Zunächst keine Codeänderungen, keine Ersetzung von Box3D oder des Zerstörungsgraphen, kein GPU-Rigid-Body-Backend. Ergebnisse als Vorschläge mit isolierten A/B-Tests.

21.4 | Benchmarkdesign und Abnahme

Primärfall: reproduzierbarer Zehn-Minuten-Schutthaufen unter Dauerbeschuss (ca. 24.000 Trümmer, laut aktueller Dokumentation 760–860 wache Körper, 2.400–2.750 berührende Kontakte, 3,9–4,3 ms Physikschritt in einem gemessenen Abschnitt). Zusätzlich: lokale Einzelgranate, Massenzerstörung, große statische Stadt, viele gemeinsame Kontaktnachbarn und wiederholtes Auftauen.
Pro Änderung: Release-Build, gleiche Hardware, gleiche Seeds/Inputs, gleiche Workerzahl, gleiche Material- und Solverparameter; Warmup und mehrfacher Lauf. Messen: CPU-Physik-Step, Broadphase, Narrowphase, Manifold-Vorbereitung, Solver/CCD, Actor-Splitting, Impact, p95/p99, Arbeitsspitzen, Speicher, aktive Kontakte/Bodies und Arbeit pro Thread. Größere Szene allein beweist keinen größeren Solver-Lastfall. Keine erfundenen Speedups; Abweichungen durch Physikqualität/Settings offenlegen.

21.5 | Lieferformat für Claude

Erzeuge eine priorisierte Tabelle: (A) bestehender Nebenan-Code, (B) fremder Quellcode und Mechanismus, (C) präzise Differenz, (D) betroffene Hotspot-Metrik, (E) Determinismus-Risiko, (F) Implementierungsaufwand, (G) A/B-Test, (H) Status „neu / schon da / getestet und verworfen“. Abschließend nur die drei am besten begründeten neuen Hypothesen nennen. Spielregeln erhalten: keine verschwundenen Trümmer, keine unbemerkt deaktivierten Kollisionen, keine künstlich in der Luft fixierten Stücke, kontrollierter statt unnötig totaler Gebäudekollaps.

Quellen: https://github.com/Holz231/Nebenan/tree/physik-8 | https://github.com/NVIDIA-Omniverse/PhysX/tree/main/blast | https://github.com/NVIDIA-Omniverse/PhysX/tree/main/physx/source

22 | CPU-Physik, deterministische GPU-Option und Entscheidungsregel

22.1 | Ist-Zustand: authoritative CPU-Simulation

Nebenan rechnet Fraktur, Bonds, Kollapsentscheidungen, Box3D-Rigid-Body-Physik, Kontaktlösung, Schlafen und Trümmeraktivierung auf der CPU. Die GPU rendert die Ergebnisse; sie entscheidet gegenwärtig nicht verbindlich über Kollisionen oder Kollaps. Die Bibliothek beansprucht bitidentische Frakturmuster über Workerzahlen und unterstützte Plattformen, dokumentiert weitere bitgleiche Tests für ausgewählte Optimierungen. Dieses Ziel muss durch fortlaufende vollständige Zustands- und Replay-Tests abgesichert werden; die Roadmap behauptet keine unabhängig bestätigte universelle Bitidentität aller Simulationspfade.

22.2 | PhysX-GPU: leistungsfähig, aber andere Garantie

PhysX kann Broadphase, Kontakte und Teile des Rigid-Body-Solvers auf geeignete GPUs verlagern. Das kann bei sehr vielen gleichzeitig aktiven Bodies/Kontakten skalieren. NVIDIA garantiert damit jedoch keine bitidentischen GPU-Resultate über beliebige GPU-Modelle, Treiber und Plattformen; Optionen wie Enhanced Determinism sind keine solche Garantie. Asynchrone Ausführung, atomare Operationen, unterschiedliche Reihenfolgen, Fließkomma- und Reduktionsunterschiede sowie CPU/GPU-Synchronisation sind Risiken. Auch die Übernahme von PhysX-Code wäre keine automatische Lösung.

22.3 | Wäre deterministische GPU-Physik grundsätzlich möglich?

Ja, theoretisch. Ein eigens konstruierter Solver könnte z. B. ganzzahlige/festkommaarithmetische Zustände, fest definierte Sortierung der Kontaktpaare, deterministische Insel-/Farbverarbeitung, feste Iterations- und Reduktionsreihenfolgen, definierte Überläufe und versionskontrollierte Rechenpfade verwenden. Das erfordert eigene Architektur, sorgfältiges Kollisionsdesign und aufwendige Validierung auf verschiedenen Geräten. Determinismus ist nicht allein durch Fixed Point, identische Seeds oder dieselbe GPU-Marke garantiert. Ein GPU-Prototyp dürfte erst nach bitidentischen Replay-/Hash-Vergleichen für alle Zielplattformen autoritativ werden.

22.4 | Warum CPU vorerst sinnvoller bleibt

Die aktuelle CPU-Pipeline ist bereits auf lokale Aktivität, statische Gebäude, inaktive Frakturstücke, eingefrorenen Schutt, Streaming und begrenzte dynamische Bodyzahlen optimiert. Entscheidend ist die aktive Kontaktlast, nicht die nominelle Häuser- oder Chunkzahl. GPU-Transfers, Synchronisation und gemischte CPU/GPU-Kontakte könnten bei kleinen, unregelmäßigen Zerstörungsereignissen teurer sein als der bestehende CPU-Step. Ein selbstgebauter deterministischer GPU-Solver würde zusätzliche Entwicklungs-, Portierungs- und Debugkosten erzeugen und die derzeitigen Plattformziele gefährden. Kein Enginewechsel ohne klaren gemessenen Gewinn bei gleichbleibender Simulation und Determinismus.

22.5 | Große Szenen: zuerst echte Engpässe unterscheiden

Mehr Gebäude ≠ mehr aktive Physik. Für weitgehend statische Städte vorrangig GPU-Frustum-/Occlusion-Culling, Render-Seiten/HLOD, instanziertes oder indirektes Zeichnen und sparsame Transform-Uploads testen; solche Visualisierungsoptimierungen verändern die authoritative Simulation nicht. Erst bei vielen simultan fallenden, kollidierenden Stücken einen separaten Solver-Benchmark erwägen. PhysX-GPU-Leistungsversprechen sind kein Ersatz für Messwerte im Nebenan-Workload.

22.6 | Verbindliche Architektur- und Freigaberegel

CPU bleibt Quelle der Wahrheit. Die GPU darf Zeichnung, Partikel, Licht, sichtbare Flüssigkeitsoberflächen und rein kosmetische Simulation übernehmen, solange ihr Ergebnis niemals authoritative Schaden-, Kollaps-, Kontakt- oder Save/Replay-Zustände beeinflusst. Bereits in Abschnitt 08/09 geplante GPU-Wasser-/Feuer-Simulationen müssen deshalb besonders geprüft werden: sobald Auftrieb, Schub, Feuer-Ausbreitung oder Materialschwächung

gameplaywirksam sind, ist eine deterministische CPU-Autorität oder nachweisbar deterministische Kopplung erforderlich. Nicht so tun, als seien GPU-Wasser/Feuer automatisch deterministisch. Prüfen: Replay-Hash, Workerzahl, OS/CPU-Architektur, Save/Load, LAN-Lockstep, p95/p99 und Fehlerfälle.

Entscheidung: Box3D-CPU und Nebenan-Destruction beibehalten. NVIDIA Blast und PhysX-CPU als Algorithmus- und Profiling-Referenzen auswerten; GPU-Rigid-Body-Forschung nur bei nachgewiesenem CPU-Bottleneck und ohne Abstriche bei deterministischen Korrektheitskriterien.

23 | Worst-Frame-Spike-Recorder und Ursachenanalyse

Ergänzung 12. Oktober 2026 | Geplante Profiler-Erweiterung | Referenz: physik-8

23.1 | Bereits vorhanden – nicht neu entwickeln

Nebenan misst bereits durchschnittliche und maximale Impact- und Physik-Step-Zeiten. In benchmark/main.c werden für Stresstest und Großstadt-Test die Zeiten pro Frame getrennt für Impact, Box3D-Step, World-Update und ggf. Streaming gesammelt, sortiert und als Mittelwert, p95 und Maximum ausgegeben; der Stresstest berichtet zusätzlich p99 der Gesamtzeit sowie Frames über 16,7 ms. Die Demo in demo/demo.cpp zeigt FPS, 1%-Low sowie Mittel und Spitze aus 300 Frames für Bild, Einschläge, Simulation und Grafik. Diese vorhandene Instrumentierung erweitern, nicht ersetzen.

23.2 | Ziel: seltene Hänger sicher erfassen

Zusätzlich ein abschaltbarer Spike-Recorder, der auch seltene p99,9-Ausreißer und Worst-Frames reproduzierbar erfasst. Maximal die 20 langsamsten Frames eines Messlaufs behalten, nicht nur einen rollenden Mittelwert. Pro Fund die exakt gleichen Ereignis-/Frame-IDs ausgeben, damit Lastspitzen unter Beschuss, Kollaps, Debris-Freeze/Thaw und schnellem Stadtstreaming eindeutig zugeordnet werden können. Ein Frame zählt nur einmal; die Messung soll nicht von laufender Sortierung oder Dateiausgabe gestört werden.

23.3 | Was je Spike aufgezeichnet wird

Zeit: Frameindex, Simulations-Tick, gesamte CPU-Framezeit, optional Render-/Present-Zeit, Box3D-Step, Collision/Broadphase/Narrowphase, Contact-Preparation, Solver/CCD, Nebenan-Impact/Fraktur, Actor-Splitting, nbWorld_Update, Streaming/Prepare/Restore/Save, Renderer-Upload und GPU-Zeit soweit zuverlässig verfügbar.
Last: Anzahl Impacts und ihre Typen, erzeugte/aktivierte/deaktivierte Bodies, erstarrter und aufgetauter Schutt, wachende Bodies, aktive und berührende Kontakte, neu erzeugte Kontakte, Anzahl Chunks/Bonds, eingefügte Shapes, verschobene/geladene/entladene Häuser, Workerzahl, Task-Anzahl, Cache-Hits/-Misses und Speicher/Allokationen (wo messbar).
Reproduzierbarkeit: Szene, Seed, Input-Skript, Build/Commit, CPU/GPU, Worker-Konfiguration, Fragmentskala, Debris-Budget und relevante Solver-Settings. Keine ungemessenen Felder als gemessene Tatsachen ausgeben.

23.4 | Kostenarm und deterministisch implementieren

Die Profiler-Daten dürfen keine Simulationsentscheidungen beeinflussen. Vorallokierte Ringpuffer und ein kleiner Top-20-Speicher statt ungebremstem Loggen; keine Dateien, Lock-Contention oder dynamischen Speicherallokationen im kritischen Simulationspfad. Instrumentierung an definierten CPU-Zeitpunkten; Thread-Zeiten getrennt erfassen, ohne sie fälschlich zur Wall-Clock-Zeit zu addieren. Die Sortierung, Detailaggregation und Ausgabe nach dem Lauf erledigen. Instrumentierungs-Overhead über identische Runs mit/ohne Recorder prüfen. Bitidentische Replay-/State-Hashes und Ereignisreihenfolge müssen unverändert bleiben.

23.5 | Reproduzierbare Benchmarkmatrix

Den bestehenden 10-Minuten-Schutthaufen unter Dauerbeschuss, einzelne Treffer, mehrere gleichzeitige Einstürze, massenhaftes Schutt-Auftauen, 100/300/1.000 statische Häuser, die 65.536-Häuser-Streaming-Szene sowie Rückkehr zu beschädigten Gebäuden testen. Release-Build, definierter Warmup, identische Seeds, Schussabfolgen, Workerzahlen und Einstellungen; mehrere Wiederholungen. Pro Phase p50, p95, p99, p99,9, Maximum sowie Anzahl Frames über 8,33 / 16,67 / 33,33 ms ausgeben. Nach CPU-Simulationsspikes und tatsächlichen visuellen Frame-Pacing-/GPU-Spikes unterscheiden; p99,9 bei zu kurzen Läufen nur mit entsprechender Stichprobe interpretieren.

23.6 | Auswertung, Claude-Auftrag und Abnahme

Claude soll zuerst die vorhandenen Messpfade im aktuellen Branch mit Funktions- und Zeilenverweisen dokumentieren und nur fehlende Erfassung hinzufügen. Danach die 20 schlimmsten Frames mit

Ursachen-Rangfolge als lesbaren Bericht und optional CSV/JSON für Diagramme ausgeben. Jeder Spike erhält zugehörige Zahlen statt Vermutungen (z. B. 340 eingefrorene Bodies und 1.100 Prüfungen statt nur „Schutt teuer“). A/B-Messungen gegen Baseline: Durchschnitt darf sich nicht relevant verschlechtern; p99/p99,9 und Maximalspitzen sollen nachweisbar sinken. Jede spätere Optimierung muss neben Performance auch Physikqualität, Replay-Hashes und Persistenz der Trümmer bestehen.

Quellstellen: github.com/Holz231/Nebenan/blob/physik-8/benchmark/main.c (Stresstest ca. Z. 453–533, Stadt ca. Z. 1008–1108); demo/demo.cpp (FPS-Meter, 300-Frame-Historie, Performance-Menü); docs/Optimierungen.md (bereits untersuchte Lastspitzen).

24 | Weitere Referenzsysteme für Optimierungen

Ergänzung 12. Oktober 2026 | Rechercheauftrag für Claude | Baseline: physik-8

24.1 | Vergleichsebenen klar trennen

Box3D ↔ PhysX, BEPUphysics v2, Project Chrono, ReactPhysics3D, MuJoCo: Starrkörperphysik, Kontakte, Kollisionssuche, Solver und Gelenke. Nebenan ↔ NVIDIA Blast, Voro++: Bruchgeometrie, Verbindungen, Schäden, Trennung und Aktivierung. PositionBasedDynamics und SPlisHSPlasH: zusätzliche spezialisierte Methoden für zukünftige Spielsysteme und deren Kopplung an Nebenan/Box3D. Diese Einteilung ist eine Arbeitszuordnung, keine Gleichsetzung des gesamten Funktionsumfangs. PhysX und Blast sind verschiedene Komponenten; Nebenan verwendet Box3D als Physik-Backend.

24.2 | Referenzliste und konkrete Forschungsfragen

System	Vergleich	Relevante Untersuchungen
BEPUphysics v2	Box3D	SIMD-Batching, SoA-Datenlayout, Kontaktverarbeitung, Constraint-Solver, Multithreading
Project Chrono	Box3D	Multicore-Broad-/Narrowphase, dichte Kontakte, parallele Solver, Speicherbandbreite
ReactPhysics3D	Box3D	Persistente Kontaktmanifolds, Convex-Kollision, Warm-Starting, Sleeping und Profiling
Voro++	Nebenan	Voronoi-Zellenschnitt, Nachbarsuche, lokaler Aufbau, numerische Robustheit
MuJoCo	Box3D / später	Gelenke, Aktuatoren, Kontakt- und Artikulationssolver für Giant, Ragdolls und Figuren
PositionBasedDynamics	Beide / später	PBD/XPBD für Hook-Seile, Stoff, flexible Stäbe und Constraints; Zweiwegekopplung
SPlisHSPlasH	Nebenan / später	SPH-Nachbarsuche, Fluid-Rigid-Coupling, Kräfte auf Schutt; MLS-MPM nicht ersetzen

Zusätzlich bereits vorgesehen: NVIDIA PhysX (Box3D-Referenz), NVIDIA Blast (Nebenan-Referenz), NVIDIA Flow (zukünftiges Feuer/Rauch). Kapitel 21–23 nicht duplizieren, sondern deren Anforderungen übernehmen.

24.3 | Priorisierung

Bestehende Destruction-/Physiklast: 1. BEPUphysics v2 für SIMD und Constraint-Batching; 2. Project Chrono für Multicore-Kontakte; 3. ReactPhysics3D für persistente Kontaktmanifolds; 4. Voro++ für lokale Voronoi-Verfeinerung. Diese Reihenfolge ist eine Rechercheprioritär, kein Geschwindigkeitsranking. Neue Gameplay-Systeme: MuJoCo für Giant/aktive Ragdolls; PositionBasedDynamics für Hooks, flexible Stäbe und Stoff; SPlisHSPlasH als Ergänzung zur geplanten Wassersimulation.

24.4 | Speziell PBD / XPBD – was ist daran interessant?

Bei klassischen Impuls-/Kraftverfahren entstehen Positionsänderungen aus Bewegungsgrößen und aufgebrachten Kräften bzw. Impulsen. PBD korrigiert Positionen direkt, um Zwangsbedingungen (z. B. Seillänge oder Biegung) zu erfüllen; XPBD erweitert dies um ein robusteres Steifigkeits-/Compliance-Modell. Das ist vor allem für Hook-Seile, flexible Stäbe, Cloth, Giant-/Worm-Segmente interessant – nicht als pauschaler Ersatz für Box3Ds konvexe Trümmerphysik. Für Seile an bewegten Trümmern sind deterministische, bidirektionale Kräfte/Impulse und stabile Kopplungsgrenzen zu untersuchen. Visuelle Plausibilität darf nicht mit vollständiger physikalischer Genauigkeit verwechselt werden.

24.5 | Verbindlicher Claude-Auftrag, Messung und Abnahme

Für jedes Projekt Quellcode statt nur README lesen und konkrete Algorithmen mit Funktionen/Dateien belegen. Vor jedem Vorschlag docs/Optimierungen.md, extern/README.md, benchmark/main.c und aktuelle

Implementierung von physik-8 prüfen. Umgesetzte, verworfene oder bereits geplante Maßnahmen markieren statt erneut zu empfehlen. Für verbleibende Ideen Engpass, erwartete Richtung ohne erfundene Prozente, Integrationsaufwand, Lizenz-/Abhängigkeitsfragen, Determinismusrisiko und isolierten A/B-Test ausweisen.

Beispiele der A/B-Tests: sustained rubble / aktive Kontaktzahl / Solverzeit; Worst-20-Frames inklusive p95, p99, p99,9 und Maximum (Kap. 23); lokale Voronoi-Zellzeit mit exakt gleicher Geometrie; später deterministische Seil- und Gelenkszenen mit identischen Inputs. Bitidentische Replay- und State-Hashes über Workerzahlen und unterstützte Plattformen sind harte Abnahmebedingungen. Keine GPU-Starrkörperphysik und kein Einfluss nichtdeterministischer GPU-Wasser-/Effektdaten auf authoritative Physics. Zunächst nur Forschungsbericht; keine Implementierungsänderung ohne gesonderte Entscheidung.

24.6 | Repository-Links für Claude

BEPUphysics v2: https://github.com/bepu/bepuphysics2

Project Chrono: https://github.com/projectchrono/chrono

ReactPhysics3D: https://github.com/DanielChappuis/reactphysics3d

Voro++: https://github.com/chr1shr/voro

MuJoCo: https://github.com/google-deepmind/mujoco

PositionBasedDynamics: https://github.com/InteractiveComputerGraphics/PositionBasedDynamics

SPlisHSPlasH: https://github.com/InteractiveComputerGraphics/SPlisHSPlasH

25 | Ergänzende Referenzsysteme: Bewegung, Fahrzeuge, Audio und deterministische Kopplung

Status: Forschungs- und Architekturpunkte ab Roadmap-Kapitel 24; keine bestehende Implementierung als fertig voraussetzen. Neu ist nicht ein Ersatz von Box3D oder Nebenans Zerstörungslogik, sondern die gezielte Untersuchung fehlender Subsysteme. Alle Vorschläge sind zuerst am aktuellen GitHub-Stand und den Profiling-/Determinismusregeln aus Kap. 21–24 zu prüfen.

25.1 | Charaktersteuerung und Parkour – Godot CharacterBody3D als Architektur-Referenz

Referenz: https://github.com/godotengine/godot (CharacterBody3D, move_and_slide und begleitende Kollisions-/Bewegungslogik). **Zuordnung:** neues Gameplay-/Controller-System in Nebenan, mit Box3D als Kollisions- und Starrkörper-Backend; keine Übernahme der Godot-Engine erforderlich.

- **Konkreter Nutzen:** robuste Steuerung für Sprint, Sprung, Landung, Slide, Wallrun, Vault, Kanten-/Stufenerkennung und Bewegung auf beweglichen oder einstürzenden Plattformen; sauberer Übergang zwischen kinematischer Figur und physikalischem Ragdoll.
- **Claude untersuchen lassen:** Trennung von Eingabewunsch, Kollisionstest, Bodenklassifikation und resultierender Bewegung; Sweep-/Shape-Casts, Slide-Projektion, Boden-Normale/Steigungsgrenzen, Snap-to-ground, Step-up/Step-down, Kontakt mit bewegten Plattformen und Behebung von Eck-/Kantenfällen.
- **Nebenan-spezifische Fälle:** zerstörte Treppen, neu entstandene lokale Deckenrampen (Kap. 17), rotierende Trümmer, verschwindender Boden unter den Füßen, Sprung aus Wallrun auf instabile Plattform und Kollision mit dynamischen Rigid Bodies.
- **Grenzen:** CharacterBody3D ist eine Godot-Klassenarchitektur, keine direkt passende Box3D-Erweiterung. Verfahren adaptieren und mit Box3D-Queries vergleichen. Deterministische Kollisionsreihenfolge/Normalenauswahl und definierte Interaktion mit dynamischen Bodies sind notwendig.
- **A/B-/Abnahmetest:** reproduzierbare Parcours-Szenen mit 1000 identischen Eingabesequenzen; Messung von CPU-Zeit, Verhaken, Penetration, Bodenverlust, Fehltritten, Replay-/State-Hashes und Verhalten bei zerstörten Flächen. Fokus auf spielbares M1-Prototype.

25.2 | Fahrdynamik – Project Chrono Vehicle als spezialisierte Referenz

Referenz: https://github.com/projectchrono/chrono (Vehicle-Module). **Zuordnung:** Fahrzeug-Gameplay oberhalb des Box3D-Backends; Chrono wird als Algorithmus-/Modellreferenz geprüft, nicht als zusätzlicher vollständiger Physiksolver vorausgesetzt.

- **Konkreter Nutzen:** dynamische Reifenkräfte (Längs-/Seitenführung), Bremsen und Drift, Aufhängung/Feder-Dämpfer, Radlasten, Schwerpunkt/Lastwechsel sowie Kollisionen bei hoher Geschwindigkeit – passend zum geplanten Burnout-inspirierten Fahrgefühl.
- **Claude untersuchen lassen:** unterschiedliche Reifenmodelle und deren Kosten, vereinfachte versus detaillierte Rad-Boden-Kontaktmodelle, Federungsintegration, Anti-Roll-Verhalten, Untersteuern/Übersteuern und Wechselwirkung mit unebenem oder zerstörtem Untergrund.
- **Integration:** zunächst günstiges Fahrzeugmodell mit Box3D-Rigid-Body, Rad-Sweeps/Raycast-Kontakten und anwenderfreundlichem Handling. Leichtgewichtige lokale Meshverformung, ablösbare Karosserieteile und Crash-Impulse bleiben getrennt von der Fahrdynamik; keine generelle FEM-/Softbody-Pflicht.
- **A/B-/Abnahmetest:** definierte Slalom-, Vollbremsungs-, Sprung-, Drift- und Bordstein-/Trümmerstrecken; Latenz, Stabilität, Radkontakt-Flattern, CPU-Zeit, Kollisionen und deterministischer Replay über Workerzahl/Plattform bewerten. Nur notwendige Modellkomplexität übernehmen.

25.3 | Zerstörungsabhängiger Raumklang – Steam Audio

Referenz: https://github.com/ValveSoftware/steam-audio. **Zuordnung:** neues Audio-/Akustiksystem in Nebenan, unabhängig vom Box3D-Kollisionssolver.

- **Konkreter Nutzen:** Schallabschattung durch Wände/Decken, veränderte Hörbarkeit nach einem Durchbruch, Reflexionen und Nachhall in Räumen, die durch Zerstörung geöffnet werden. Optional bessere Richtungs- und

Distanzwahrnehmung bei Explosionen, Schritten, Fahrzeugen und Gegnern.

- **Claude untersuchen lassen:** Geometrie-Export für akustische Szenen, dynamische Geometrieaktualisierung, Occlusion-Queries, Reflexions-/Reverb-Kosten, Material-/Absorptionsdaten und die Trennung zwischen schnellen approximierten Effekten und aufwendigerer akustischer Berechnung.
- **Nebenan-spezifische Integrationsidee:** akustische Geometrie in gröberen, streambaren Clustern statt jedes Voronoi-Trümmerstück als vollständiges Audio-Mesh zu behandeln. Bei lokalem Wanddurchbruch nur betroffene Cluster invalidieren; kleine Trümmer können per Material-/Occlusion-Näherung behandelt werden.
- **Grenzen:** Audio ist grundsätzlich nicht autoritativ für Box3D; unnötige Rebuilds und Framespitzen verhindern. API-/Plattform- und Lizenzbedingungen bei einer tatsächlichen Integration vorab prüfen.
- **A/B-/Abnahmetest:** identisches Haus vor/nach Wanddurchbruch, mit/ohne Trümmer, gleichzeitigen Explosionen und mehreren Schallquellen; Audio-Update-ms, Speicher, Spikes und hörbare Übergänge messen. Keine Kopplung von Audioresultaten an deterministische Spielzustandsentscheidungen.

25.4 | Deterministische Systemkopplung und Netzwerkzustand – Architektur statt Bibliothek

Zuordnung: Integrationsarchitektur zwischen Box3D (autoritative Physik), Nebenan (Schäden/Bruch/Streaming), geplanten Wasser-/Feuersystemen, Terrain, NPCs, Fahrzeugen und optional Multiplayer. Hier vorerst **keine zusätzliche Bibliothek** empfehlen: erst Rechen- und Synchronisationsgrenzen definieren.

- **Autoritative Reihenfolge festlegen:** feste Timestep-Phasen für Input, Damage/Fracture, Structural Updates, Body-Aktivierung, Box3D-Step, Kontakt-/Ereignisübernahme und Streaming. Ereignisse stabil sortieren; IDs, Seeds, Worker-Scheduling und Reduktionsreihenfolgen explizit festlegen.
- **CPU/GPU-Grenze:** GPU-Wasser/Feuer/Rauch darf als nichtautoritativer visueller Effekt laufen. Sobald Wasserkräfte auf Trümmer oder Feuer Materialfestigkeit und Einsturz beeinflussen, braucht diese Rückkopplung ein deterministisches, autoritatives CPU-Modell oder eine nachweislich plattformstabile Alternative; GPU-Readbacks nicht ungeprüft in den State übernehmen.
- **Streaming, Savegame, LAN:** kanonische Serialisierung aller relevanten Fragment-/Bond-/Rigid-Body-/Terrainzustände, feste Wiederherstellungsreihenfolge, Replay mit Input-/Event-Log, periodische State-Hashes und gegebenenfalls Snapshot-/Rollback-Strategie untersuchen. Split-Screen ohne Netzwerk von LAN-Determinismus getrennt testen.
- **Spike- und Replay-Tests:** zeitgleiche Hochlastfälle (großer Gebäudeeinsturz + Brand + Flut + Fahrzeuge + NPC-Pfade), p95/p99/p99,9/max pro Phase, Top-20-Spikes (Kap. 23), CPU/GPU-Übergänge, deterministische Replay-Hashes bei 1/N Workern und unterstützten Plattformen. Erst nach isolierter Profilierung konkrete Optimierungsmaßnahmen vorschlagen.

25.5 | Priorität und verbindlicher Claude-Arbeitsauftrag

Reihenfolge nach Entwicklungsnutzen: 1. Character-Controller/Bewegungsarchitektur für M1; 2. deterministische Systemkopplung früh konzipieren, bevor GPU-Wasser/Feuer Gameplayzustände beeinflussen; 3. Chrono Vehicle bei Umsetzung der Fahrzeuge; 4. Steam Audio bei Audio-Implementierung. Dies ist ein Roadmap-Fokus, keine Behauptung bereits gemessener Performancevorteile.

Claude soll die konkreten relevanten Quellcodedateien der Referenzen lesen und **für jeden Kandidaten** die Differenz zur aktuellen Nebenan-Implementierung, erwartete Funktionalität, Schnittstellen zu Box3D/Nebenan, Determinismus- und Lizenzrisiken sowie einen kleinen, reproduzierbaren Testplan dokumentieren. Bereits vorhandene Lösungen in README, docs/Optimierungen.md und Roadmap prüfen, keine doppelten Vorschläge erzeugen. Zunächst Research und Integrationsentwurf, keine ungefragten Architekturwechsel.

26 | Hybrid-Animation und physische Trefferreaktionen im Overgrowth-Stil

Ergänzung 14. Oktober 2026 | Verbindliche Designrichtung für Character/Combat M1–M2. Erweitert Kapitel 02, 03, 15, 19 und 25. Bisherige Inhalte bleiben unverändert; geplante Features werden nicht als implementiert dargestellt.

26.1 | Architekturentscheidung: „Fake it Realism“ statt vollständiger Körper-KI

Hauptweg ist ein **Overgrowth-inspirierter Hybrid**: regulärer Character Controller für präzise, schnelle Eingaben; Referenzanimationen mit Blending/Inertialization für die Kernbewegung; selektive additive Reaktionen, IK und bei schweren Kontakten aktive bzw. passive Ragdoll-Physik. Überzeugende Treffer entstehen aus Geometrie, Reaktionsrichtung, geeigneten Clips und Impulsen - nicht aus einem durchgehend neuronalen Ganzkörpercontroller.

Bestehende Festlegungen bleiben gültig: Warframe-artige Bewegung soll unmittelbar steuerbar sein (Kap. 02); Ragdoll ist primär für schwere Treffer, Stürze und Skate-Bails vorgesehen. Leichte Treffer dürfen per prozeduraler Pose-Korrektur sichtbar nachgeben, ohne die volle Ragdoll-Simulation für jede Figur dauerhaft zu aktivieren.

26.2 | Animationen aus Video - Offline-Produktionspipeline

- **Aufnehmen:** eigene vollständige Bewegungsclips filmen (Idle, Start/Stopp, Sprint, Slides, Tritte, Schwertangriffe, Blocks, Stolpern, Stürze, Aufstehen). Gute Ausleuchtung und möglichst wenig Verdeckung; mehrere Perspektiven nur mit passender synchronisierter Rekonstruktion.
- **Rekonstruieren:** Claude Code automatisiert Videoimport, zeitliche 3D-Pose-Schätzung (z. B. GVHMR), Qualitätsindikatoren und einen Bild-/Skelettvergleich. Claude muss nicht selbst jeden Videoframe als Winkel schätzen.
- **Bereinigen:** Knochenlängen/Gelenkgrenzen, zeitliche Glättung, Boden- und Fußkontakte, Root-Motion und erkennbare Occlusion-Fehler prüfen. Schwierige Bewegungen gezielt manuell korrigierbar halten.
- **Retargeting:** auf die Skeleton-first-Rig-Namen, Achsen und Proportionen aus Kap. 19 abbilden; Animationen mit Kontakt- und Ereignismarkern für Trefferfenster, Fußaufsatz und Recovery exportieren.
- **Laufzeit:** Clips in einer einzigen Animation State Machine / Bewegungsbibliothek verwalten. Aufträge wählen Bewegungen aus; Blending, Inertialization und IK passen Übergänge und Kontaktpunkte an.

Eine physikalisch erlernte Balance ist **keine Voraussetzung** für die Animationserstellung. Das vorhandene Rig liefert separat Kollisionskörper, Gelenkgrenzen und Ragdoll-Zustände.

26.3 | Einbindung in Nebenan und Box3D

Keine neue allgemeine Physik-Engine und keine parallele autoritative Figurphysik einführen. Der Kinematic-/Hybrid-Controller aus Kap. 02 bleibt für reguläre Bewegung zuständig; bei aktivierter Ragdoll werden Körpersegmente und Gelenkconstraints mit der vorhandenen Box3D-CPU-Physik gekoppelt. Vorher prüfen, welche Joint-Motoren und Kopplungen der aktuelle Branch tatsächlich unterstützt; fehlende Funktionen isoliert ergänzen.

26 | Reaktionssystem, Ragdoll und Recovery

26.4 | Trefferreaktionen nach Stärke und Kontext

Die folgende Matrix ist ein **Designziel**, keine starre Einteilung nur nach Schadenszahl. Trefferpunkt, normalisierte Richtung, relative Geschwindigkeit, Impuls, Haltung, Bodenlage und gerade laufende Aktion fließen in die Auswahl ein. Ein geblockter Schlag reagiert anders als ein ungedeckter Treffer.

Trefferlage	Angestrebte Reaktion / Technik
Leichter Tritt	Additive Pose-Reaktion von Brust, Schulter und Kopf; gedämpftes Abklingen in die laufende Animation. Füße bleiben im Stand.
Mittlerer Tritt	Gerichteter Ausgleichsschritt aus kleinem Reaktionsclip; Fußziel via IK und Bodenprüfung anpassen, Oberkörper zeitversetzt zurückführen.
Starker Tritt	Stolper-/Bail-Animation einleiten; bei überschrittener Stabilitäts- oder Treffergrenze kontrolliert in vollständige Ragdoll übergehen.
Extremer Tritt	Sofortiger Ragdoll-Übergang mit Kontaktpunkt, Richtung und abgestimmtem Knockback-Impuls; reale Weltkollision und Schwerkraft.
Nach dem Sturz	Ruhe-/Kontaktzustand prüfen, Rücken- oder Bauchlage erkennen, passend ausgerichteten Aufstehclip starten und zurück in die normale Steuerung blenden.

26.5 | Technische Schichten in der Runtime

- **Core Animation:** Idle/Locomotion/Attack/Dodge sowie Übergänge über Blending und Inertialization; Angriffstempo, Timing, Telegraphing und Eingaberesponsivität bleiben kontrollierbar.
- **Procedural Reaction:** lokale additive Rotation/Translation von Oberkörper, Kopf und Armen; Gewichte sowie Dämpfung je Körperpartie und Trefferlage, nicht nur ein globaler Physik-Prozentwert.
- **Fuß-/Bodenlogik:** reaktive Schritte aus wenigen Schritt- oder Stolperclips wählen, anschließend root-orientieren und mit IK, Foot Lock und Umgebungstests platzieren.
- **Ragdoll State:** bei Fall/Kontrollverlust die aktuelle Pose sowie lineare und Winkelgeschwindigkeiten möglichst sprungfrei an die Box3D-Körper übergeben. Physik bestimmt danach Kontakt, Rollbewegung und Aufprall.
- **Recovery State:** ruhige Lage prüfen, standfeste Aufstehfläche finden, passende Bauch-/Rücken- oder Seitlage wählen; mit Pose-Matching und kollisionsgeprüfter Root-Position zurück zum Controller.

26.6 | Schwert-, Kick- und Umweltkopplung

Hitboxes/Sweeps müssen dem animierten Körper bzw. der Waffenbahn folgen. Physische Treffer benötigen nachvollziehbare Kontaktpunkte und relative Bewegung; der Treffer übergibt Impuls-, Schaden- und Materialinformationen an die gemeinsame Nebenan-Impact-Schnittstelle (Kap. 03/14). Klinge gegen Klinge: Block/Parry/Bind abhängig von Angriffsbahn und Kontaktgeometrie statt garantierter Auto-Hits. Bei Fehlschlägen trifft der Impuls gegebenenfalls Objekte oder zerstörbare Umgebung.

Gameplay darf Impulse bewusst überzeichnen („Fake it Realism“), muss aber Richtungen, Massewirkung, Kollisionsreaktionen und die visuelle Übernahme der Körpergeschwindigkeit konsistent darstellen. Besonders starke Treffer wechseln in passive Ragdoll, anstatt einen stets gewinnenden Balance-Controller zu erzwingen.

26 | Umsetzung, Tests und optionale Forschung

26.7 | Konkrete Build-Reihenfolge und Abnahme

- **M1 / Rig-Basis:** einen automatisch geriggten Humanoid mit Capsule-Kontrolle, Idle, Gehen und Angriff inklusive Übergängen, sauberem Fußkontakt und Debug-Skelett darstellen.
- **M2a / Trefferreaktionen:** Kräfte gezielt an Brust/Schulter/Kopf testen; leichte additive Reaktion und mittlerer Ausgleichsschritt funktionieren unabhängig von einer Gegner-KI.
- **M2b / physischer Fall:** starker Treffer führt reproduzierbar in Ragdoll, mit korrektem Impuls, ohne Pose-/Velocity-Sprung; Kollisionen mit Boden, Wand und Schutt funktionieren.
- **M2c / Recovery:** Figur beruhigt sich, erkennt ihre Lage, steht mit passendem Clip ohne Bodendurchdringung auf und übernimmt wieder Player-/NPC-Steuerung.
- **Erweiterung:** Schwertkontakt, Parieren, Skate-Bail, Fahrzeuge und Giant/Fighter (M7) über die gleichen Reaktions-/Impact-Events anbinden; archetypspezifische Massen und Clips separat parametrieren.

26.8 | Performance, Determinismus und A/B-Test

CPU-autoritative Treffer, Ragdoll-Körper, Kontaktereignisse und Zustandsübergänge müssen die Replay-/Determinismusregeln aus Kap. 22/23 erfüllen. Rein kosmetische additive Animation kann lokal sein, darf aber keine unsynchronisierten Damage- oder Physics-Entscheidungen auslösen. Fixierte Tick-Reihenfolge, stabile Event-IDs, feste Schwellen/Zustandslogik und vorhersehbare Aktivierung/Deaktivierung der Physikkörper vorsehen.

Vergleichen: 1, 10 und 50 NPCs; Idle, wiederholte Treffer, gleichzeitige Stürze, Aufstehen, zerbrechende Deckung sowie Szene mit aktivem Debris. CPU-Zeiten getrennt für Animation, IK, Ragdoll, Box3D-Kontakte, Damage/Impact; p95/p99/Max, RAM und Replay-Hashes pro Test dokumentieren. Keine behaupteten Speedups ohne Messung.

26.10 | Quellen und verbindlicher Claude-Auftrag

Overgrowth-Quellcode: github.com/WolfireGames/overgrowth (Architektur-/Implementierungsreferenz, keine 1:1-Übernahme). Video-Rekonstruktion: github.com/zju3dv/GVHMR (Lizenz und Pipeline-Eignung vor Übernahme prüfen).

Claude soll zuerst den aktuellen Nebenan-Branch und die bestehenden Kapitel 02/03/14/19/22/23/25 prüfen, dann nur fehlende Komponenten, Schnittstellen und A/B-Tests spezifizieren. Ziel ist ein reproduzierbarer Prototyp: Angriff + leichter/mittlerer/starker Treffer + Ragdoll-Fall + Aufstehen. Erst nach erfolgreicher Abnahme weitere Bewegungsarten.

26 | Overgrowth-Quellcode: geprüfte Mechanismen

Ergänzung 14. Oktober 2026 | Präzisierung zu 26.1–26.6. Befund aus öffentlichem Overgrowth-Code; Übertragung auf Nebenan ist ein eigener Entwicklungsauftrag. Keine bereits implementierte Funktion wird behauptet.

26.11 | Bestätigt: Animationen sind das Fundament

Befund (Overgrowth): Die Charaktersteuerung spielt echte Clips ab, z. B. `r_idle.anm`, Treffer- und Aufsteh-Clips. `SetAnimation` und `SetCharAnimation` wechseln Clips mit Übergangsblende; `AddLayer` ergänzt weitere Posen. Die Bewegung wird über XML-Blendtrees und Variablen wie `speed_coord`, `tall_coord` und `threat_coord` gemischt.

Befund (Overgrowth): IK ist eine nachgeschaltete Korrektur: Fußkontakt, Hüfte, Brust, Kopf, Blick und Arme werden aus einer bereits berechneten Animationspose angepasst. Der Code kompensiert sogar Verschiebungen des Schwerpunktes, die durch Kopf-/Brust-Korrekturen entstehen. Das ist vereinfachte Balancehilfe, keine universelle Schritt-Recovery.

Für Nebenan: Animationsclips und Blendtrees bleiben verbindlicher Standard für Idle, Sprint, Parkour, Angriffe, Blocks und getroffene Reaktionen. Prozedurale IK und additive Layers korrigieren die Clips; sie ersetzen die Animationsbibliothek nicht. Die Offline-Video/Mocap-Pipeline aus 26.2 bleibt unverändert.

26.12 | Bestätigt: Treffer sind teils Event- und Regel-basiert

Befund (Overgrowth): Die Animation erzeugt zeitlich definierte Ereignisse wie `attackimpact`. Der Angreifer prüft Reichweite und Zielzustand; `WasHit(...)` wählt Block, Ausweichen, animierte Reaktion oder Ragdoll. Das Gameplay benutzt dabei teilweise eine gezielte Positionierungshilfe („Magnetisierung“). Es ist also **keine** durchgehend rein physikalische Hand-/Fuß-gegen-Körper-Simulation.

Befund (Overgrowth): `Reaction::GetAnimPath(severity)` wählt Trefferanimationsvarianten nach Schwere. Bei starken Treffern entsteht die Kraft aus Angriffsdaten (u. a. `GetForce()`), Verteidigerzustand und Knockback-Multiplikator. `ApplyForceToRagdoll()` verteilt diese gewichtet nach Körpermasse und Distanz zum Trefferpunkt auf die Ragdoll-Bodies.

Für Nebenan: Animationsereignisse markieren Angriffsphasen, jedoch wird im freigegebenen Trefferfenster mit kinematischen Sweeps/Hitvolumen von Bein, Hand oder Klinge geprüft. Ein Event allein darf keinen garantierten Treffer auslösen. Trefferdaten gehen als einheitliches Kontakt-/Impulsereignis an Character und World Impact. Spielhilfen wie Attack Magnetism bleiben optional und konfigurierbar.

26.13 | Bestätigt: aktive Ragdoll besitzt Animations-Reflexe

Befund (Overgrowth): Ragdoll-Modi sind nicht nur „an/aus“. `_RGDL_FALL` nutzt u. a. `r_flail.anm` sowie Schutz-Layers `r_catchfallfront.anm` und `r_fetal.anm`. `_RGDL_INJURED` verwendet `r_writhe.anm`. Die Stärke der Gelenkkontrolle ändert sich mit Geschwindigkeit, Zustand und Bewusstlosigkeit; passive Ragdolls erhalten Gelenkstärke null.

Für Nebenan: Zusätzlich zur Impulsübernahme 2–3 billige Schutz-Layers vorsehen: Arme abstützen, Kopf schützen, zusammenrollen. Ihre Gewichte anhand Fallrichtung, Nähe zu Hindernissen und verfügbarer Zeit wählen. Keine Garantie, dass jede Reaktion den Aufprall verhindert: Box3D-Kontakte bestimmen den tatsächlichen Verlauf.

26 | Gezielt ergänzen: Balance, Recovery, Weltschaden

Dies sind geplante Nebenan-Erweiterungen des Overgrowth-Vorbilds. Die Regel- und Physiksteuerung bleibt gameplayorientiert.

26.14 | Ergänzung: reaktiver Ausgleichsschritt statt sofortigem Sturz

Status: Overgrowth besitzt Fuß-IK und Schwerpunktkorrekturen, aber aus dem geprüften Code folgt **kein** universeller, physikalisch gelernter Step-Recovery-Controller. Für Nebenan wird eine kleine deterministische Heuristik ergänzt.

Eingaben: Projektion des Schwerpunkts auf den Boden, Körpergeschwindigkeit, aktuelle Fußkontakte/Standfläche, Trefferimpuls, Charaktermasse, Bodenbeschaffenheit, Collider-Freiraum und laufende Aktion. Nicht allein die Schadenszahl entscheidet.

Logik: 1) Standfuß und Stützfläche bestimmen; 2) Neigung/Schwung gegenüber einer abstimmbaren Stabilitätszone bewerten; 3) bei moderater Gefahr einen Rückwärts-/Seitwärts-Schrittclip wählen; 4) Schritt in die tatsächliche Impulsrichtung orientieren und mit Foot-IK, Plant-Lock und Bodenprüfung ausrichten; 5) bei blockiertem Schritt oder zu hohem Impuls stolpern bzw. Ragdoll aktivieren.

Abnahme: Seitenschub, Frontkick, Schräge, Stufe, sich lösender Boden und nahe Wand. Figur darf nicht durch Geometrie treten oder übernatürlich jede Kraft kompensieren. Schwellen mit Hysterese und reproduzierbarer Tick-Reihenfolge gegen Flattern sichern.

26.15 | Ergänzung: Ragdoll-Aufstehen mit Sicherheitsprüfung

Befund (Overgrowth): `HandleRagdollRecovery()` wartet auf geringe Bewegung und eine Recovery-Freigabe. `WakeUp()` unterscheidet Stand, Fall, Flip und Roll; `PlayStandAnimation()` wählt je nach Körperlage passende Aufsteh-Clips. Beim Übergang werden Ragdoll-Bone-Pose und Animation miteinander geblendet.

Für Nebenan: Vor Freigabe prüft die CPU-Physik Ruhegeschwindigkeit, tragfähige Fläche, freien Kopfraum, Platz für Arm/Bein und Kontakt zu Trümmern. Bei Hindernissen nicht die Figur teleportieren: Warten, Kriechen/Rollen oder erneut Hinfallen. Pose und Position anpassen, bevor die Kollisions-/Steuerautorität zurück an den Character Controller wechselt.

Fehlerfall: Ein gerade fallendes Stück Schutt oder ein zweiter Treffer bricht den Aufstehvorgang ab. Recovery ist ein überprüfbarer Zustandsübergang, keine rein kosmetische Überblendung.

26.16 | Ergänzung: Ragdoll durch eine zerstörbare Wand

Zielszene: Tritt → Impuls in Ragdoll → realer Körperkontakt an der Wand → Box3D-Kontakt und Nebenan-Impact → lokaler Voronoi-Bruch → fortgesetzter Flug durch die Öffnung → Landung unter Debris → optionales Recovery.

Kopplungsregel: Die Ragdoll-Bodies besitzen echte Kollisionshüllen, Material- und Massenwerte. Der Kontakt erzeugt maximal ein wohldefiniertes, tick- und paaridentifiziertes Impact-Ereignis je relevanter Kollision; Impuls und relativer Bewegungszustand fließen in die bereits vorhandene Damage-Pipeline. Visuelle Knockback-Überzeichnung und autoritative Bruchentscheidung bleiben getrennt. Kein „Wand bricht immer“-Skript.

Abnahme: Holz/Beton in unterschiedlichen Dicken, Treffer frontal/seitlich, mehrere Kontakte innerhalb eines Ticks, durchgebrochene Kante, Falling Debris auf liegendem Gegner. Physikalische Energie und Materialwiderstand dürfen einen Durchbruch auch **verhindern**. Keine doppelt angewendeten Impulse oder vergessenen Kollisionen.

26 | Integration, Performance und konkrete Arbeitspakete

Fortsetzung und Präzisierung von 26.7–26.10. Bestehende Box3D-/Nebenan-Architektur, CPU-Autorität und Replay-Regeln haben Vorrang vor dem Übernehmen fremder Engine-Teile.

26.17 | Physik-LOD: Animation ist der billige Normalfall

Die folgende Laststaffelung ist ein **Implementierungsziel**, keine gemessene Overgrowth-Performance. Nicht jeder NPC erhält in jedem Frame einen vollständigen aktiven Ragdoll-Solver. Physik wird bei relevanter Interaktion zugeschaltet.

NPC-Zustand	Geplante Berechnung
Fern / nicht relevant	Vereinfachte Clipauswertung, seltenere IK-Updates, reduzierte Bone-LOD; authoritative Trefferprüfung nur nach Relevanzregeln.
Normal bewegt	Animation State Machine, günstige Bodenkontrolle, IK nach Sichtbarkeit/Nähe; kinematischer Controller.
Kampf / leichte Hits	Zusätzliche Sweep-Tests, additive Trefferpose und animierte Reaktionen; kein permanenter Full-Ragdoll-Step.
Starker Hit / Bail	Aktive oder passive Ragdoll mit Box3D-Körpern und Gelenkconstraints; Schlafen bei Ruhe.
Recovery	Kurzzeitig Pose-Matching, freie Standfläche und stabile Controller-Übergabe prüfen.

Budgetvorschlag: maximal gleichzeitig voll simulierte NPC-Ragdolls nach Priorität (Spielernähe, Sicht, aktive Kollision, Quest-/Gameplay-Relevanz). Niemals Kontakte zu autoritativ bewegtem Debris bloß aus visueller Distanz deaktivieren. Erst Profiling mit 1/10/50 Figuren und Kollapsszenen; p95/p99/Maximum und State-/Replay-Hashes aus Kap. 22/23 vergleichen.

26.18 | Minimale Schnittstellen und klare Zuständigkeiten

CharacterMotion: Auswahl von Clips, Blend-Koordinaten, Animation-Layers und IK-Zielen; gibt Zielpose, Eventmarker und Root-Bewegung aus.

HitResolver: prüft Angriffsfenster mit Sweeps und Blockstatus; gibt eine eindeutige `HitEvent`-ID sowie Trefferpunkt, Normale, relativ gerichtete Geschwindigkeit, Impulsstärke, Waffe/Material und beteiligte Körper aus.

ReactionController: entscheidet anhand dieses Events plus Körperzustand: additive Reaktion, Ausgleichsschritt, Stolpern oder Kontrollverlust. Übt keine zweite, konkurrierende Physik aus.

RagdollBridge / Recovery: überträgt aktuelle Pose, lineare und Winkelgeschwindigkeit in Box3D; verwaltet aktive Gelenkstärke, Schutz-Layers, Fallzustand und kontrollierte Rückübergabe.

WorldImpactBridge: nimmt ausschließlich autoritative Box3D-Kontakte entgegen und übergibt sie mit Tick-/Event-ID an Nebenans vorhandene Impact-/Destruction-Schnittstelle. Kein zweiter Fraktur- oder Schadenssolver.

26.19 | Neue Abnahmetests und Claude-Arbeitsauftrag

Priorität A: Ein Standcharakter erhält Front- und Seitentreffer: sichtbar nachgeben, Ausgleichsschritt versuchen oder fallen, je nach messbarem Impuls und freien Fußzielen.

Priorität B: Treffer im Lauf/Wall-Run sowie Skate-Bail: Animation und Geschwindigkeit dürfen beim Ragdoll-Wechsel nicht springen. Wurf und Gegenstände bleiben kollisionsgültig.

Priorität C: Knockback gegen zerstörbare Wand: echte Kontakte, plausibler oder ausbleibender Bruch, erneuter Kontakt zu Schutt, sicheres Aufstehen oder Abbruch bei Gefahr.

Claude Code: Zuerst Joint-, Collision- und Impact-APIs im aktuellen Branch prüfen, dann nur fehlende Schnittstellen ergänzen. Overgrowth ist Referenz; Änderungen mit deterministischen Replay-Tests absichern.

26 | Quellen und Integrationsgrenzen

Quellcodebefunde geprüft am 14. Oktober 2026. Die Dateien beschreiben Overgrowth, nicht bereits vorhandene Nebenan-Funktionen. Alle Funktionsverweise sind als klickbare Fundstellen angegeben.

26.20 | Konkrete Overgrowth-Fundstellen

- Data/Scripts/aschar.as: `UpdateAnimation` (um Zeile 11834), `UpdateHitReaction` (um 9712), `HandleRagdollRecovery` (um 5002), `PlayStandAnimation` (um 9805), `HandleAnimationEvent` (um 7295), `HandleRagdollImpactImpulse` (um 6137).
- Source/Graphics/animationclient.cpp: `SetAnimation`, `GetMatrices`, `AddLayer`; animierte und überblendete Knochenposen.
- Source/Objects/riggedobject.cpp: `RiggedObject::Ragdoll`, `UnRagdoll`, `ApplyForceToRagdoll`, `SetRagdollStrength`; Animation und Bullet-Gelenkconstraints.
- Source/Asset/Asset/reactions.cpp: `Reaction::GetAnimPath(severity)` für gestufte Reaktionsclips.
- Source/Graphics/skeleton.cpp: `Skeleton::SetGFStrength` und physikalische Gelenkconstraints.
- Source/Objects/movementobject.cpp: Animations- und Ragdoll-Übergaben; native Fuß-IK-Funktionen.

26.21 | Was ausdrücklich nicht 1:1 übernommen wird

Overgrowth verwendet **Bullet Physics**; Nebenan soll die bereits etablierte modifizierte **Box3D-CPU-Physik** behalten. Bullet-Joints und IK-Funktionen sind nicht direkt austauschbar. Physikalische Körper, Gelenkantriebe, Constraint-Grenzen, Synchronisation und deterministische Reihenfolge müssen gegen den aktuellen Nebenan-Branch geprüft und neu angebunden werden.

Overgrowth verwendet für Körpertreffer teilweise Animations-Events, Reichweitenprüfungen und Zielhilfe. Für die von Nebenan gewünschte nachvollziehbare Umwelt- und Schwertphysik sind echte geometrische Hit-Sweeps, Kontaktpunkte und physikautoritative Impulse zusätzlich erforderlich. Die Overgrowth-Methode bleibt eine bewährte **Game-Feel-Referenz**, aber keine Garantie realistischer Kollisionssimulation.

26.22 | Designentscheidung bleibt unverändert

Overgrowth-inspirierte Hybridanimation bleibt der Hauptweg: eigene Clips, Blending, IK, prozedurale Balance und nur bei Bedarf aktive oder passive Ragdoll. Die normale Bewegung bleibt schnell und direkt kontrollierbar; reale Kontakte und Impulse kommen aus Box3D. Eine komplexere Controller-Architektur ist kein Bestandteil dieses Grundsystems.

Definition of Done: Drei Trefferstärken, dynamischer Ausgleichsschritt, Ragdoll-Sturz, zerstörbare Wand mit echtem Kontakt, sicheres Recovery und reproduzierbares CPU-Profiling funktionieren gemeinsam.

27 | Articulated Bodies, Joint-Motoren und Box3D-Integration

Ergänzung vom 15. Oktober 2026 | Technischer Bauplan für M1/M2/M7 | Status: geplant, nicht als implementiert verifiziert

27.1 | Einordnung und Abgrenzung zu den vorhandenen Kapiteln

Kapitel 19 definiert die Skeleton-first-Pipeline; Kapitel 26 legt den Overgrowth-inspirierten Hybrid aus Animation, IK, Trefferlogik, selektiver aktiver/passiver Ragdoll und Recovery bereits fest. Dieses Kapitel ergänzt ausschließlich den darunterliegenden Physik- und Integrationspfad. Die Kapitel 02, 03, 14, 15, 19, 21–23 und 26 sowie die bestehende Destruction-Logik bleiben gültig.

Die vorliegende Roadmap beschreibt modifiziertes Box3D als autoritative CPU-Physik und Nebenan als eigenständige Destruction-Bibliothek. Vor jeder Implementierung im aktuellen Branch prüfen, welche Joint-Typen, Constraint-APIs, Motoren, Shape-Filter, Warmstart-Pfade und Replay-Garantien tatsächlich vorhanden sind. Keine unbestätigten Implementierungsannahmen aus der Roadmap als Codebestand ausgeben.

27.2 | Grundarchitektur: dieselbe Physikwelt, keine zweite Engine

- RigAsset: unveränderliche Bone-Hierarchie, Restpose, Bone-IDs, Gelenkachsen, Grenzen und Motorprofile. Physik-Assets werden aus dem Rig abgeleitet, nicht aus Render-Polygonen.
- ArticulationDefinition: per Charaktertyp beschriebene einfache Kollisionsformen, Masse, Schwerpunkte, Trägheit, Eltern-Kind-Verbindungen und lokale Joint-Frames; zwischen gleichen Archetypen teilbar.
- ArticulationInstance: eigene Box3D-Body-/Joint-Handles, Bewegungszustand und Warmstart-Daten; keine Laufzeit-IDs in gemeinsamem Asset-Cache speichern.
- Motion/Balance/IK: berechnen gewünschte Posen und Fußziele; sie erzeugen bei aktivierter Ragdoll Motorziele und schreiben nicht parallel direkt in die physikalischen Körpertransforms.
- RagdollBridge: einzige Zuständigkeit für Aktivierung, Pose- und Velocity-Übernahme, Motorstärken, Kollisionsfilter und Rückgabe an den normalen Controller.
- WorldImpactBridge: autoritative Box3D-Kontakte erzeugen eindeutig identifizierte Impact-Events für die vorhandene Nebenan-Schadenspipeline; keine doppelte Kraft- oder Schadenanwendung.

27.3 | Zustandsmodell und Autorität

Zustand	Autoritativ für Körperbewegung	Simulation
Normal: Idle/Locomotion	Kinematischer/Hybrid-Controller; Animation + IK bestimmen die Pose.	Gering; selek
Leichter/mittlerer Treffer	Controller + additive Hit-Pose oder gerichteter Recovery-Step; keine permanente Full-Ragdoll nötig.	Zusätzliche S
Schwerer Treffer/Bail	Box3D-Rigid-Bodies mit Gelenkconstraints; aktive Gelenkmotoren können Schutz-/Erholungspose versuchen.	Aktive, geko
Passiver Fall / Ruhe	Box3D bestimmt freie Körperbewegung; Motoren aus bzw. reduziert.	Kontakte, sp
Recovery	Physik prüft stabile, freie Aufstehposition; Übergabe an Controller erst nach Pose-/Velocity-Abgleich.	Kurzzeitig er

27 | Joint-Constraints und Motorsteuerung

27.4 | Erstes Minimalexperiment

Bevor ein Humanoid gebaut wird: zwei Boxen mit einem Hinge-Joint. Danach ein dreiteiliger Arm, anschließend eine passive 12–16-Körper-Testfigur. Ziel: korrekter Massentransfer, stabile Limits, stabile Kontaktreaktionen. Erst danach motorisierte Posen, IK und aktive Balance.

27.5 | Gelenke als Solver-Constraints

Jedes Gelenk verbindet zwei Physikkörper über lokale Joint-Frames. Zur Laufzeit werden daraus die Welt-Anker und Achsen bestimmt. Fixed-Joint blockiert sechs relative Freiheitsgrade, Hinge erlaubt eine Rotation, Ball/Swing-Twist erlaubt begrenzte Rotationen. Constraint-Fehler, Jacobians, effektive Masse, Limits und Impulsgrenzen müssen zum bestehenden Box3D-Solver passen; konkrete Implementierung erst nach Quellcodeprüfung entscheiden.

- Reihenfolge: Fixed-Joint → Hinge mit Limit → Ball-/Swing-Twist-Joint → stabile Motoren → aktive Körperkette → Humanoid.
- Solver-Kopplung: Motoren, Gelenke und Bodenkontakte müssen sich gegenseitig berücksichtigen. Ein Animationsziel darf nicht nachträglich einen Körper durch Geometrie teleportieren.
- Numerik: feste Zeitschritte, toleranzbewusste Stabilisierung, konsistente Winkel-/Quaternion-Fehler; keine extremen Massenverhältnisse und keine unbeschränkten Motorimpulse.

27.6 | Joint-Motoren: Zieldrehung statt Pose-Teleportation

Für einen 1-DOF-Hinge eignet sich konzeptionell ein gedämpfter Positionsregler: Torque = Kp · Winkelfehler + Kd · Geschwindigkeitsfehler, jeweils mit Drehmoment-/Impulslimits. Für 3D-Gelenke Quaternion-/Orientierungsfehler benutzen. Bevorzugt solver-integrierte, begrenzte Motorconstraints oder eine numerisch stabile implizite Federform testen; ein naiver hochverstärkter expliziter PD-Regler kann instabil werden.

Pro Gelenk mindestens: TargetRotation, TargetAngularVelocity, Stiffness, Damping, MaxTorque, Enable/Strength und Achs-/Limitkonfiguration. Unterschiedliche Profile für Stand, Airborne, Schutzpose, starken Treffer und passive Ragdoll. Einfache Motorziele allein bedeuten noch keine Balancekontrolle.

27.7 | Per-Body-Kollisionen und Trefferpunkt

- Torso, Becken, Kopf, Ober-/Unterarme, Ober-/Unterschenkel und Füße bekommen zweckmäßige primitive Kollisionsformen mit plausibler Masse und Trägheit; Render-Low-Poly bleibt davon getrennt.
- Lokale Kontakte aus Box3D enthalten Body-ID, Position, Normale, relativen Bewegungszustand und den bereits vom Solver berechneten Impuls. Dieser Impuls darf bei der Übergabe an Nebenan nicht noch einmal als zusätzliche physikalische Kraft auf den gleichen Körper angewandt werden.
- Direkt benachbarte Körperteile bei Bedarf von Selbstkollision ausschließen; nicht alle Selbstkollisionen pauschal abschalten. Torso/Arme, Beine/Umwelt und Körper/Schutt bleiben je nach sinnvoller Konfiguration kollisionsfähig.
- Angriffe im Normalzustand nutzen deterministische Sweeps/Hitboxes; im Ragdollzustand werden reale Box3D-Bodykontakte für physische Interaktion benutzt. Ereignisse nach Tick, Paar und Kontaktphase deduplizieren.

27.8 | Übergabe von Animation an Physik

Beim Aktivieren der Ragdoll werden aktuelle interpolationsfreie Simulationspose und lineare/Winkelgeschwindigkeiten je Segment abgeleitet und in die Box3D-Instanz übernommen. Root-Geschwindigkeit, Plattformbewegung sowie Trefferimpuls konsistent behandeln. Während der Ragdoll ist nur der Physikzustand authoritative. Für Recovery stabile Unterstützung, Kollisionsfreiheit und plausible Pose prüfen, dann Root-/Bone-Zustände angleichen, Clips überblenden und die Steuerautorität in genau definierter Tickphase zurückgeben.

27 | Recovery, Flug und Integration

27.9 | Prozedurales Rebalancing ohne Vollzeit-Ragdoll

Im normalen Character-Controller liefert eine deterministische Balanceheuristik Schrittziel und Reaktionsklasse. Eingaben: Schwerpunktprojektion, COM-Geschwindigkeit, Fußkontakte, Stützpolygon, Impuls, Oberflächennormale, Hindernisse und erlaubte Beinreichweite. Ein Capture-Point-/Extrapolated-COM-Schätzer ist eine mögliche Näherung für geeignete Standphasen, kein universelles Stabilitätsgesetz bei Wandlauf, Gleitbewegung oder Luftflug.

Für moderate Störungen: reaktiven Step-Clip wählen, in Impuls-/Bewegungsrichtung ausrichten, Fußziel auf erreichbare freie Fläche begrenzen und mit Foot-IK/Plant-Lock anpassen. Für starke Störungen oder fehlenden Standplatz: Stolper-/Bail-Übergang bis zur aktiven/passiven Ragdoll. Der Charakter darf nicht jeden Schlag sicher ausbalancieren.

27.10 | Midair-Repositionierung, Kick und Landung

- In der Luft keine Bodenbalance: IK kann Pose und Gliedmaßenziele verändern, aber ohne externe Kräfte lässt sich der Gesamt-Drehimpuls nicht beliebig auf null setzen. Tuck/Extend, Torso-Twist und Armbewegung verändern Trägheit und relative Orientierung.
- Landing Predictor: Flugbahn mit eigener Root-/Physikgeschwindigkeit vorhersagen; Terrain-/Geometrie-Sweeps für wahrscheinlichen ersten Kontakt; Stance/Kniebeugung und Fußziele rechtzeitig vorbereiten.
- Arcade Assist: optionale begrenzte Root-Korrekturen/air control ausdrücklich als Gameplayhilfe ausweisen und für deterministische Replay-Pfade berechnen. Keine implizite Behauptung, es sei reine freie Ragdoll-Physik.
- Landing State: nach horizontaler Geschwindigkeit, Aufprallwinkel, Untergrund, Fußkontakt und verfügbarer Bremsstrecke entscheiden: stehen, weiterlaufen, stolpern, rollen oder ragdollen.

27.11 | Schnittstelle zur zerstörbaren Welt

Charaktere, Giants, Fahrzeuge und Gebäudeschutt bleiben Teilnehmer derselben autoritativen CPU-Kontakt- und Impact-Pipeline. Der Physikkörper eines fallenden Charakters kann dadurch echte Wandkontakte auslösen. Lokale Materialregeln, Bonds, Bruch und Kollaps folgen den bereits festgelegten Nebenan-Regeln; keine neue exklusive Charakter-Destruction-Pipeline. Gelenke des Charakters sind nicht mit den in Kapitel 17 ausdrücklich künstlich gelenklosen Bruchzonen gleichzusetzen.

27.12 | Spätere Nutzung für Giant / Worm / Skate

Giant: vergrößerte Articulation mit Motor-Torque-/Speed-Limits, Zielpunkt-IK und Standflächenlogik, wie in Kapitel 11 vorgesehen. Worm: segmentiertes Rig, aber weiterhin bevorzugt bahngeführtes Hybridmodell statt pauschal einem simulierten Körper je Segment (Kapitel 12). Skate-Bail: Übergang des bestehenden Fahr-/Character-Controllers in die identische RagdollBridge. Gemeinsame Low-Level-Constraints wiederverwenden; Steuerstrategien bleiben archetypspezifisch.

27 | Optimierung, Benchmarks und Arbeitsauftrag

27.13 | Optimierung nur nach Profiling

Priorität	Maßnahme / Prüfbedingung
A	Self-Collision-Filter für ungeeignete Eltern-Kind-Paare; tatsächliche Kontaktkosten und Körperüberschneidung prüfen.
A	Warmstart und persistente Joint-/Kontaktimpulse, sofern im aktuellen Box3D nicht bereits vorhanden; Replay-Reihenfolge absichern.
A	Konstante Joint-Frames und Assets cachen; keine runtime Body-IDs in Archetyp-Definitionen speichern.
A	Sleeping und Aktivierungs-/Deaktivierungslogik; bei relevantem Debris-/Gameplaykontakt niemals rein optisch wegoptimieren.
B	Constraint-Islands, Cache-Lokalität und Batch-Solverpfade prüfen; existierende Box3D-Optimierungen nicht duplizieren.
B	LOD für Animation/IK/Bones und Ragdoll-Aktivierungsbudget; Sichtbarkeits-LOD von autoritativer Kontaktpflicht trennen.
Später	Reduced-coordinate articulated-body solver, SIMD oder Multithreading nur bei identifiziertem Engpass und reproduzierbaren A/B-Gewinnen untersuchen.

27.14 | Messmatrix und Abnahmekriterien

Stufen: (1) Hinge-Pendel, (2) 3-Link-Arm, (3) passive 12–16-Bodies-Testfigur, (4) motorisierte Figur, (5) normaler Idle/Walk → Impact → Recovery, (6) 1 / 10 / 50 NPCs in aktiver Destruction-Szene. Jede Stufe zuerst in kleinen Tests, danach im gemeinsamen Welt-Benchmark.

- Stabilität: maximale Joint-Anchor-Trennung, Limit-Überschreitung, Restschwingungen, NaNs, Collider-Penetration, ungewollte Selbstkollision und Latenz bis zum Sleep.
- Charakter: Trefferrichtung, Fußziel-Freiraum, Recovery-Erfolg/Misserfolg, Velocity-Kontinuität beim Bail, kein Clip-Teleport, reaktive Landung und Ragdoll-Aufstehen unter Trümmern.
- Performance: getrennt messen: Animation, IK, Joint-/Motor-Setup, Contact/Narrowphase, Solver, RagdollBridge, WorldImpact, Streaming und Gesamt-Frame. p50/p95/p99/p99,9, Max, RAM und Worst-Frame-Recorder (Kapitel 23).
- Determinismus: identische Seeds/Inputs/Worker-Szenarien, State-/Replay-Hashes und definierte Update-Reihenfolge. Keine nichtdeterministischen Entscheidungen aus GPU-Visuals ableiten (Kapitel 22).

27.15 | Konkrete Build-Reihenfolge für Nebenan

Phase	Lieferung / Done
Vorarbeit: Branch-Audit	Aktuellen Box3D-/Nebenan-Code gegen Kap. 21–26 prüfen; vorhandene Joint-, Motor- und Kollisionspfade dokumentieren; fehlende APIs markieren.
P1: Physik-Basis	Hinge/Fixed/Ball bzw. vorhandene Äquivalente und Limits im bestehenden Solver stabil; Zwei-Boxen-Demo unter Kontaktbelastung.
P2: Motoren	Gedämpfte, begrenzte Joint-Motoren; Pendel und Drei-Link-Arm stabil, ohne harte Pose-Teleports.
P3: Physikskelett	Aus Kapitel-19-Rig abgeleitete Bodies/Joints; passive humanoide Ragdoll mit dedupizierten Welt-Impacts.
P4: Hybrid-Bridge	Animation/IK-Zielpose → Motors bei Bedarf; pose- und velocity-kontinuierliche Übergänge; Recovery sicher.
P5: Gameplay	Kapitel-26-Impulse, 1–2 Recovery-Steps, Airborne-Pose/Landing und zerstörbare Wand; 1/10/50-NPC-Benchmarks.

27.16 | Verbindlicher Claude-Code-Arbeitsauftrag

Zuerst tatsächlichen Branchzustand und docs/Optimierungen.md lesen; vorhandene Joint-, Motor-, Broadphase-, Filter-, Event-, Solver-, Multithread- und Replay-Pfade mit konkreten Code-Fundstellen belegen. Erst danach pro fehlender Funktion eine kleine, isolierte Änderung vorschlagen. Architektur nicht ersetzen, keine parallele Physik einführen, Benchmarks vor und nach jedem Eingriff vergleichen. Probleme oder nicht vorhandene APIs klar benennen; keine gemessenen Speedups behaupten. Die Redaktionsregel von Seite 1 bleibt erhalten.

28 | Overgrowth-Hybrid: Steuerungsübergaben

Präzisierung vom 15. Oktober 2026 | Ergänzung zu Kapitel 26/27 | geplant, nicht implementiert

28.1 | Genau eine Autorität je Simulationszustand

Normalbetrieb: Der Character Controller bestimmt Root-Bewegung und Kollisionen; Animation und IK erzeugen die visuelle Pose. Bei aktiver Ragdoll bestimmen Box3D-Körper, Gelenke und Kontakte die tatsächliche Bewegung; Clips und IK liefern nur Motorziele. Im passiven Fall folgen die Körper allein der Physik. Kein gleichzeitiges direktes Schreiben in denselben Root- oder Bone-Transform durch mehrere Systeme. Zustandswechsel erfolgen an einer definierten Fixed-Tick-Grenze.

28.2 | Trefferreaktion: visuelles Feedback von physischer Wirkung trennen

Ein Treffer wird über eine eindeutige HitEvent-ID ausgewertet. Leichte und mittlere Treffer können über additive Posen, Root-Reaktion, Schrittclip und IK umgesetzt werden, ohne dass ein vollphysikalisches Skelett läuft. Erst bei Ragdoll-Aktivierung übernimmt der Kontakt-/Impulspfad die Körperphysik. Der ursprüngliche Trefferimpuls darf beim Wechsel nicht doppelt angewendet werden. Treffer ohne Ragdoll dürfen trotzdem echte Impulse auf Umweltobjekte auslösen; Character- und World-Impact bleiben getrennt zuordenbar.

28.3 | Ausgleichsschritt: IK platziert, aber entscheidet nicht

Die Balanceheuristik prüft Unterstützung, Schwerpunktbewegung, Impuls, Fußkontakte und erreichbare freie Schrittziele. Sie entscheidet, ob und wohin ein Fuß gesetzt wird. Ein Schritt-/Stolperclip gibt Rhythmus und Gewichtsverlagerung vor, IK korrigiert das Fußziel, Plant-Lock verhindert Gleiten. Danach muss die neue Standfläche tatsächlich geprüft werden. Bei blockiertem Ziel, fehlender Auflage oder zu hoher Restgeschwindigkeit folgt ein weiterer Schrittversuch oder ein Bail; nicht automatisch ein garantiertes Abfangen.

28.4 | Ragdoll-Rückübergabe ohne Positions- und Geschwindigkeitssprung

Beim Umschalten Pose sowie lineare und Winkelgeschwindigkeit der Körpersegmente übernehmen; bewegliche Plattformen und Kontaktimpulse berücksichtigen. Beim Aufstehen zuerst freie Aufstellfläche und anhaltende Stabilität nachweisen, dann den Root zur physikalischen Lage ausrichten, die Ausgangspose zum Aufstehclip matchen und die Physikautorität kontrolliert an den Character Controller abgeben. Bei Trümmerkontakt, engem Raum oder erneuter Störung Recovery abbrechen oder verzögern.

28.5 | Neue kleine Abnahmereihe (vor komplexen NPC-Szenen)

(A) Stand + Treffer links/rechts mit gleichem Impuls: reproduzierbare Reaktion. (B) Schrittziel durch Wand blockiert: kein Fußteleport. (C) Treffer exakt beim Umschalten: kein doppelter Impuls. (D) Bail auf beweglicher Plattform: Geschwindigkeitskontinuität. (E) Aufstehen unter Schutt: keine Durchdringung. Zu jedem Test Tick-/Event-IDs, Root-/COM-Verlauf, Kontaktstatus, Joint-Fehler und Replay-Hash erfassen; vorhandenen Spike-Recorder verwenden.

29 | Angriffswunsch statt sofortiger Schlag

Ergänzung 15. Oktober 2026 | zu Kapitel 03, 26 und 28 | geplant, nicht implementiert

29.1 | Grundprinzip und Eingabe

Gedrückte Angriffstaste (z. B. linke Maustaste) signalisiert Angriffsabsicht, nicht zwingend einen sofortigen Angriff. Solange die Taste gehalten wird, darf das Kampfsystem eine geeignete Aktion starten, sobald ein gültiges Ziel in passender Reichweite und ein zulässiges Zeitfenster erreicht sind. Loslassen beendet die vorgemerkte Absicht; ein begonnener, bereits verbindlicher Angriff wird dadurch nicht willkürlich abgebrochen. Das Prinzip ist an Overgrowth angelehnt, seine konkrete Ausgestaltung für Nebenan ist eine Designentscheidung.

29.2 | Kontextauswahl ohne Treffer-Garantie

Die Auswahl berücksichtigt Abstand und relative Bewegung, Blick-/Zielrichtung, Bewegungszustand (Stand, Sprint, Slide, Luft, Hocke), freie Schlagbahn, Aktionssperren sowie Waffe bzw. unbewaffneten Stil. Geeignete Clips sind etwa Punch, Kick, Sweep oder Lufttritt. Reichweite löst die Aktion aus, nicht den Treffer. Die anschließende Hit-Erkennung erfolgt über tatsächliche, zeitlich geführte Körper-/Waffen-Sweeps oder physische Kontakte. Gegner können ausweichen, blocken; Angriffe können verfehlen.

29.3 | Explizite Umweltangriffe und Spielerkontrolle

Da Nebenan eine Zerstörungs-Sandbox ist, dürfen Wände, Türen, Schutt, Objekte und leere Luft nicht grundsätzlich vom Angriff ausgeschlossen sein. Neben dem kontextabhängigen Hold-Angriff einen klaren direkten Angriffspfad anbieten, z. B. separater Input, Modifikator oder expliziter Zielmodus. Keine automatische Zielmagnetisierung durch Wände, keine unverlangten Richtungswechsel und keine Garantie, dass ein Schlag ein Objekt trifft. Zuordnung der Tasten erst im Input-Design festlegen.

29.4 | Deterministische Zustandsmaschine

Vorschlag: Idle → AttackIntent → Windup → Committed → ActiveHitWindow → Recovery → Idle. In AttackIntent Zielvalidierung pro Fixed-Tick; ausgewählte Aktion und Ziel-ID zu einem festen Commit-Zeitpunkt einfrieren. Nach Commit begrenztes Retargeting, damit Angriffe lesbar und ausweichbar bleiben. Bei fehlendem Ziel bleibt die Absicht aktiv, jedoch ohne endlose Neuauslösung während Recovery/Cooldown. Input, Zielauswahl, Hit-IDs und Aktivierungs-Tick replay- und determinismusfähig halten.

29.5 | Anschluss an vorhandene Nebenan-Systeme

Die Aktionswahl speist die Animation/Blendtree-Logik; IK korrigiert Zielpunkte und Standfüße, ersetzt aber keine Angriffskollision. Der Character Controller behält in normalen Angriffen die Root-Autorität. Treffer laufen durch das gemeinsame Impact-/Schadensereignis in Box3D und Nebenan-Destruction. Bei starken Treffern übernimmt die in Kapitel 28 definierte Ragdoll-Übergabe. Ein Impact wird nicht gleichzeitig als animierter Zusatzimpuls und als physischer Kontakt doppelt eingetragen.

29.6 | Prototyp und Abnahme

Tests: (1) Taste aus Distanz halten, Angriff startet erst bei gültiger Reichweite; (2) Gegner weicht nach Commit aus → sauberer Fehlschlag; (3) Sprint/Slide/Sprung führen zu passenden Aktionen ohne abrupte Geschwindigkeitsnullung; (4) direkter Schlag gegen Wand ohne Gegner funktioniert; (5) blockierter Schlagpfad und Cooldown werden beachtet; (6) identische Eingaben/Seeds erzeugen gleiche Attack-/Hit-Events; (7) Split-Screen und hohe NPC-Zahl ohne auffällige p95/p99-Spitzen.

30 | Sifu × Overgrowth: physikbasiertes PvP-Kampfsystem

Ergänzung 17. Oktober 2026 | zu Kapiteln 03, 19, 22, 26–29 | Designziel, noch nicht implementiert

30.1 | Zielbild und Abgrenzung

Nebenan erhält einen reaktiven Nahkampf für Spieler gegen Spieler und KI: die defensive Präzision und enge Distanz aus Sifu kombiniert mit den glaubwürdigen, impulsabhängigen Körperreaktionen von Overgrowth. Ein Ausweichen ist primär eine Kopf-, Rumpf-, Hüft- oder Beinbewegung innerhalb der Nahkampfdistanz, kein automatischer Sprung aus der Reichweite. Treffen sollen sich durch Position, Impuls, Stützfläche und Umweltkontakt auswirken. Die Konzepte ergänzen den bestehenden Nahkampf (Kapitel 03) und ersetzen weder den Character Controller noch den CPU-/Box3D-Unterbau.

Designentscheidung: Balance, Haltung, Struktur, Verletzungs- und Funktionszustand sowie der reale Aufprall haben spielmechanischen Vorrang vor einem klassischen Lebensbalken. Lebenspunkte können für bestimmte Missionen optional bleiben; der Kernkampf muss ohne reines HP-Abarbeiten funktionieren. Auch ein starker Treffer kann fehlschlagen, abgefangen oder durch einen Stabilisierungsschritt überstanden werden.

30.2 | Drei Angriffsklassen mit eigener Bedeutung

Klasse	Zweck und Konterlogik
Striking	Schläge, Ellenbogen, Knie und Kicks wirken durch Reichweite, Winkel, Kontakt und Impuls. Schnelle Treffer können riskante Greif-/Intercept-Versuche unterbrechen.
Grappling	Eigenständige Antwort auf passives Blocken: Kontakt herstellen, Hand/Arm/Körper kontrollieren, Clinch, Ziehen, Drücken oder Wurf. Benötigt Reichweite, greifbaren Körperteil und stabiles Timing.
Intercepting	Eigenständige Antwort auf einen bereits begonnenen Dodge: gezieltes Abfangen der momentanen Ausweichbahn mittels Abdrängen, Körperkontrolle, Knie/kurzem Treffer oder Positionsbruch. Kein bloßer automatischer Folgeschlag.

Für Intercepting ist keine Finte und kein vorheriges Raten als Pflichtmechanik vorgesehen: Ein Spieler erkennt die tatsächlich laufende Ausweichbewegung und reagiert innerhalb eines kurzen Fensters mit einem separaten Input. Intercept ist kein Auto-Hit; Reichweite, Stand, Ausweichrichtung, Timing und physischer Kontakt müssen stimmen. Wird ohne gültigen Dodge abgefangen, entstehen Recovery und Gegenangriffsrisiko.

30.3 | Körpernahes Ausweichen bei Schlägen und Kicks

Die Ausweichrichtung orientiert sich an der tatsächlichen Bedrohung: Ducken unter hohen Treffern, Slips links/rechts gegen gerade Schläge, Lean-back gegen kurze Schlagbahnen, Hüft-/Rumpfrotation gegen Hooks, Beinheben oder Gewichtsverlagerung gegen Low Kicks. Ein Low-Kick-Dodge erfordert glaubwürdigen Fußkontakt bzw. kurzzeitige Einbeinbalance. Diese Animationen dürfen die Figur nicht teleportieren oder aus dem Kontaktbereich herausziehen.

Defensivaktionen verändern die Körperhülle, Trefferflächen und Schwerpunktlage. Ein erfolgreiches Ausweichen eröffnet eigene Konterfenster; ein schlecht getimtes kann von Intercepting abgefangen werden. Ein Grab darf nicht alle Dodges pauschal schlagen. Der jeweilige Angriff muss den tatsächlich aktuellen Körper erreichen.

30.4 | Reaktives PvP und faire Gegenmaßnahmen

Situation	Verlässliche Designantwort – keine garantierte Wirkung
Block wird gehalten	Gezielter Grab/Clinch umgeht Deckung bei erfolgreichem Griffkontakt.
Dodge hat bereits begonnen	Intercept prüft die reale Ausweichrichtung; erfolgreicher Kontakt stört Balance/Position.
Grab oder Intercept kommt zu früh	Schneller Strike, Abstand oder Richtungswechsel kann unterbrechen bzw. ins Leere führen.
Dodge ist korrekt getimt	Verteidiger behält Nahkampfdistanz und erhält ein kurzes Konterfenster.
Parade gelingt	Angriff wird abgelenkt; Initiative, Strukturwirkung und Folgemöglichkeit entstehen aus Timing und Kontakt.

30.5 | Balance, Struktur und physischer Knockdown

Der Körperzustand umfasst mindestens Standfläche und Fußkontakte, Schwerpunkt/COM samt Geschwindigkeit, Körperausrichtung, aktuelle Bewegung, aktive Kontakte, Struktur/Guard-Stabilität, lokale Funktionsbeeinträchtigung und Erholungsstatus. Ein Schlag erzeugt einen Impuls und ggf. Strukturverlust; der Balance-Controller versucht abhängig von Impulsrichtung, Haltung, vorhandener Standfläche und freien Schrittzielen die Figur zu stabilisieren. Misslingt das, folgt Stolpern, Knockdown oder aktive/passive Ragdoll. Kein willkürlicher Sturz allein wegen eines Schadenswerts.

Ein sauberer Kick kann den Gegner zurückstoßen, gegen eine Wand treiben oder umwerfen – aber nur, wenn Kontakt, Größenordnung und Richtung der Wirkung sowie verbleibende Stabilität das unterstützen. Masse, Momentum, Bodenreibung, Hindernisse und Gegenbewegung wirken mit. Wände, Türen, Schutt und Fahrzeuge können reale Kontakt- und Destruction-Ereignisse auslösen. Knockdowns sind kein garantierter Effekt einer bestimmten Taste.

30.6 | Verletzungen statt reiner HP-Schwellen

Ein leichtgewichtiger Zustandsansatz unterscheidet temporäre Trefferbenommenheit, Einschränkung einzelner Körperbereiche, Strukturbruch und kampfentscheidende Funktionsunfähigkeit. Beinschäden verschlechtern Stand und Kick-Stabilität; Armschäden beeinträchtigen Griffkraft oder Deckung; Rumpftreffer beeinflussen Gleichgewicht und Recovery. Die Effekte bleiben lesbar und balancierbar, statt eine unberechenbare anatomische Vollsimulation zu verlangen. Für Sandbox-Modi sind Intensität, Recovery und Verletzungsmodell konfigurierbar.

30.7 | Architektur: Controller, Animation/IK, Physik

Im Normalzustand bleiben Root und Fortbewegung beim Hybrid-Character-Controller. Animation liefert die beabsichtigte Bewegung, IK präzisiert Hände/Füße/Kontaktziel, die Kollisionsschicht validiert die tatsächlichen Sweeps und Kontakte. Bei Instabilität übernimmt die in Kapitel 28 spezifizierte Active-Ragdoll-/Box3D-Physik. Motorziele lenken Gelenke, erzwingen jedoch keine physikalisch unmöglichen Endposen. Bei Recovery wird die Physik autoritativ und ohne Pose-/Velocity-Sprung wieder an den Controller übergeben.

Grappling und Intercepting verwenden gemeinsame Kontakt- und Constraint-Daten (z. B. kurzlebige Hand-/Arm-Anbindung oder kontrollierte Joint-Ziele), keine starre geskriptete Zwei-Personen-Teleportanimation. Griffabbruch bei Reichweitenverlust, Hindernis, fehlender Balance oder erfolgreichem Gegenkonter. Kein zweiter Rigid-Body-Solver, keine doppelten Hit-Impulse; Treffer über

eindeutige HitEvent-IDs an die gemeinsame WorldImpact-Schicht koppeln.

30.8 | PvP-Synchronisation, Determinismus und Spielgefühl

Split-Screen und LAN waren in Kapitel 01/14 bisher offene Ziele; Online-PvP ist hier eine neue optionale Erweiterung und keine behauptete bereits vorhandene Funktion. Für Netzwerk-PvP muss eine autoritative Simulation über Input-/Tick-IDs, Action-Commit, Kontakt- und HitEvent-IDs, Griffe, COM/Struktur, Gelenkzustände und Ragdoll-Übergänge entscheiden. Lokale Vorhersage/Interpolation darf Treffer, Verletzung oder Knockdown nicht selbst verbindlich bestimmen. Replays und Divergenztests sichern Zustandskonsistenz; Lag-Compensation ist nur mit reproduzierbarem Kontaktstand sinnvoll.

Input-Latenz, Timingfenster, Hitstop, Kamera, Antizipation, Strike-/Grab-/Intercept-Recovery und Lesbarkeit der Körperbewegungen müssen priorisiert getestet werden. Physikalische Vielfalt ist erwünscht, zufällige oder unlesbare Niederlagen nicht. Parameter zuerst als prototypische Tuningwerte behandeln, keine unbelegten exakten Millisekunden oder Trefferchancen festschreiben.

30.9 | Prototyp, Akzeptanztests und Roadmap-Anschluss

PvP-Vertical-Slice: Zwei gleich große humanoide Figuren in einer kleinen Testarena mit Punch, Kick, High-/Low-Dodge, Block, Grab, Intercept, Balance-Step, Knockdown und Recovery. Dazu eine Wand und ein bewegliches Hindernis. In dieser Reihenfolge ergänzen: (1) reproduzierbare Strike- und Dodge-Kollisionen; (2) Grapple vs. Block; (3) Intercept während Dodge samt Fehlversuchen; (4) COM-basierter Stolper-/Ragdoll-Übergang; (5) Wandimpact/Destruction; (6) Verletzungswirkungen; (7) Split-Screen-/LAN-/Netztests nach Scope-Freigabe.

Abnahmefälle: Ducken vermeidet nur passende Schlaghöhe; Low Kick kann Beinheben treffen/verfehlen abhängig von Geometrie; Block ist greifbar, aber Grab kann verfehlen; Intercept reagiert nach Dodge-Beginn und funktioniert nicht als garantierter Homing-Treffer; Kick bei gutem Stand führt zu Stabilisierung, bei schlechter Stütze zu Sturz; Wandaufprall löst genau ein Impact-Ereignis aus; Aufstehen unter Schutt scheitert ohne Durchdringung; Replay-Hashes und Fixed-Tick-Ereignisreihenfolge sind nachvollziehbar. p95/p99-Framezeiten mit 2/10/50 Kämpfern einschließlich Umgebung messen.

Einordnung: Im bestehenden M2 zunächst lokaler Basiskampf, Dodge-/Grapple-/Intercept-Logik und physische Trefferreaktionen; Kapitel 19 und 26–28 liefern Rig und Hybrid-Brücke. PvP-Balance, Verletzungssystem und eine mögliche Online-Architektur sind separate Ausbaupakete vor einer verbindlichen Launch-Zusage. Der Funktionsumfang der 1.0-Roadmap bleibt ohne ausdrückliche Neupriorisierung unverändert.